

The Adaptive Company

A Practical Operating System for Human-Agent Work

Gabriel Sarmiento

Contents

	How to Read This Book	5
	Preface: The Company Is the Design Problem	7
I	Part I: The Constraint Moved	
1	When Production Gets Cheap	11
2	Hidden Operational Debt	17
3	Why Tools Do Not Change the Operating Model	23
II	Part II: The ATOM Model	
4	ATOM: Intent In, Evidence Out	27
5	Context Is Infrastructure	31
6	Design the Decision Before the Agent	33
7	Human-Agent Execution	35
III	Part III: How the System Runs	
8	A Week Inside an Adaptive Company	39
9	Governance in the Work	43
10	Observe, Prove, Learn	45
IV	Part IV: Build the First Loop	
11	Choose a Decision, Not an AI Project	49
12	Build the Minimum Viable Operating Loop	51
13	Prove Value Before You Expand	53
V	Part V: Scale Without Losing the Plot	

14	Fractal Units, Not Fractal Bureaucracy	57
15	Allocate Capital, Talent, and Knowledge Dynamically	59

VI	Part VI: Leadership and Control
----	--

16	Agent Operations: Control at Scale	63
17	The Manager After Coordination Gets Cheap	65
18	Accountability Cannot Be Delegated	67
	Appendices	69
	Field Kit	71
	Bibliography	75

How to Read This Book

This book is built around one operating problem: AI increases the amount of analysis and execution a company can produce, but production is rarely the only constraint. Decisions, context, interfaces, control, and learning determine whether that new capacity becomes value.

Part I identifies the shift and shows how Hidden Operational Debt can be observed in customer conversations, workflow records, software, and decision traces. Part II defines ATOM - the Adaptive Transformation Operating Model - as a loop from intent through evidence and learning. Part III shows the loop operating with real stops, exceptions, and human judgment. Part IV gives a bounded path from pilot selection through a thirty-day build and a ninety-day evidence gate. Parts V and VI address multi-business scale, shared resources, agent operations, management, and accountability.

The book is explicit when introducing research evidence, a real documented practice, an illustrative example, or an operating hypothesis. Those distinctions matter. A useful model should not borrow certainty from a company story or present a plausible scenario as evidence.

You do not need to implement the whole model. Choose one recurring decision that crosses a painful boundary. Read the book with that decision in mind. By the Field Kit, you should be able to specify its intent, context, authority, execution, evidence, controls, and learning cycle on one page.

Preface: The Company Is the Design Problem

There is a point in a growing company when being the person who can solve anything becomes a liability.

The founder-COO has hired strong people. A Head of Operations now runs delivery. Product, finance, and commercial leaders own their functions. Yet the consequential decisions still return to the founder: a non-standard price, a senior hire, a promise to a strategic customer, a conflict between the established business and a new one.

The company has delegated tasks. It has not transferred the context and authority required to make decisions.

AI makes this condition easier to see and easier to worsen. Teams can produce analysis, software, campaigns, and operating plans much faster. Agents can increasingly act in business systems. But the company still has to decide what matters, which evidence to trust, where authority sits, and who answers when the result is wrong. Faster production sends more work toward any unresolved boundary.

This book is for the operating leader at that boundary: usually a founder, COO, or transformation executive helping a company move from a few dozen people toward a real management system, often while adding products, business units, or geographies. The problem is not simply adoption. Most readers already use AI. The problem is building a company that can convert human and machine intelligence into sound decisions without routing every exception through one person.

ATOM - the Adaptive Transformation Operating Model - is the model developed in these pages. It is a hypothesis about how such a company can operate. It joins intent, context, decisions, human-agent execution, evidence, and learning under explicit controls. It is not a software product, an org chart, or a promise of autonomous management.

The standard throughout the book is practical. Where a claim rests on research, the source is named. Where a company practice is public, it is identified as a real case. Where the narrative combines familiar patterns, it is labeled as an illustrative example. The reader should be able to disagree with the model, alter it, and still use its parts.

The central claim is simple: when production becomes cheap, company design becomes the constraint. The founder's next job is not to make more decisions. It is to design a company capable of making them.

I

Part I: The Constraint Moved

1	When Production Gets Cheap	11
1.1	A real change, stated narrowly	11
1.2	Productivity belongs to a system	12
1.3	The constraint moves	13
1.4	Cheap creation has an option cost	15
1.5	Won't agents absorb the new queues?	15
1.6	What to do on Monday	16
2	Hidden Operational Debt	17
2.1	The company is leaving a trail	17
2.2	Debt has an observable fingerprint	18
2.3	Real practice: agent builders trace the path	18
2.4	The dashboard should show strain, not activity	19
2.5	Illustrative example: a new business line seen through its traces	19
2.6	Ask AI to build the first version	20
2.7	From reporting to an operating instrument	21
3	Why Tools Do Not Change the Operating Model	23
3.1	What an operating model contains	23
3.2	The paved cow path	23
3.3	Local output can create system load	24
3.4	The test	24

1. When Production Gets Cheap

A common early failure caused by cheap production is not bad output. It is a company that produces more choices than it can absorb.

Consider a founder-COO in a company of forty people. The business is preparing to add a second activity, hire another thirty people, and build its first real layer of management. A Head of Operations is taking over the daily run. This should release the COO to work on structure, capital allocation, and growth.

It does not.

The new business lead can prepare a market analysis in hours. Finance can generate several pricing and margin scenarios. Product and engineering can prototype different offers in parallel. Each team arrives better prepared. Each also arrives with a decision that crosses the boundaries of the company: which assets remain shared, which capability belongs inside the new unit, what risk the center will carry, and what evidence justifies further investment.

The Head of Operations can run the process. Only the COO still holds enough of the company in their head to resolve the tradeoffs. Work has been delegated. Decisions have not.

This is an **illustrative composite**, not a report about a particular company. Its mechanism is common enough to recognize. At forty people, the founder's intervention can be the shortest path to a good answer. During the move to seventy, then perhaps one hundred and fifty, the same habit becomes part of the architecture. Teams learn to escalate because the founder is fast, informed, and exacting. Managers learn that initiative is welcome until it conflicts with an unstated preference. Shared functions accumulate exceptions. New ventures remain attached to the person who launched them because nobody designed the conditions for transfer to the run.

AI can intensify this dependency. It lowers the effort required to produce an analysis, option, plan, prototype, or recommendation. It does not automatically distribute the authority to choose among them. The organization has not shortened the path from intent to result. It has moved the queue toward judgment, integration, and commitment.

1.1 A real change, stated narrowly

Speed has no organizational meaning until we specify where it occurs and what constrains the complete result. A model can answer in seconds while the company still moves through the same decision, integration, and acceptance cycles. The useful question is not "How fast is AI?" It is "Which constraint moves when this work becomes cheaper?"

The defensible claim is narrow. The marginal cost and latency of producing *some* cognitive and software artifacts have fallen sharply. The emphasis belongs on *some*. The effect varies by task, quality threshold, workflow, and consequence.

One useful indicator is the price of model inference at a fixed level of benchmark performance. Stanford's 2025 AI Index reported that the price for a model performing around the GPT-3.5 level on MMLU fell from about \$20 per million tokens in November 2022 to \$0.07 in October 2024—a decline of more than 280 times [1]. Epoch AI, whose data informed that analysis, found very different rates across tasks and performance thresholds: roughly ninefold to nine-hundredfold annual declines in the cases it examined [2]. The spread matters. There is no single "price of intelligence," and the fastest historical decline should not be projected indefinitely.

Inference price is also not workflow cost. A reliable system may require data access, retrieval, tools, orchestration, evaluation, security, human review, and recovery from failure. A cheap model call can sit inside an expensive operating process.

Economist Joshua Gans offers a useful discipline for thinking about that difference. He treats AI prediction as an input to a decision and examines its value alongside substitutes, complements, automation, and system effects [3]. The point for an operator is straightforward: making one input cheaper does not make the complete decision cheap. The firm still needs data, judgment, and the capacity to act.

Modern generative systems stretch the prediction framing because they can produce artifacts and, when connected to tools, execute actions. The economic discipline still holds. As execution becomes cheaper, its complements become more important. A company needs to specify who may commit resources, what

constraints cannot be bypassed, what evidence must return, and how a bad action can be stopped or reversed. These are not features of the model. They are features of the operating system around it.

Capability has expanded as well. METR’s task-horizon research shows agents completing increasingly long, well-specified tasks in software engineering, machine learning, and cybersecurity [4]. But the researchers warn against turning this into a claim about whole jobs. Their benchmark is concentrated in technical tasks with clear scoring and relatively little organizational context. A 50 per cent success rate is not dependable operation. Messy work, tacit knowledge, competing goals, and institutional consequences change the problem.

The difference between adoption and autonomy is equally important. Stanford’s 2026 AI Index reports AI use in 88 per cent of surveyed organizations, while deployment of agents remains in the single digits across nearly all business functions [5]. The survey does not show that 88 per cent of companies have redesigned work or captured value. It shows that contact with AI is broad while delegated execution remains early.

These findings justify attention. They do not justify declaring the autonomous company inevitable.

1.2 Productivity belongs to a system

“Does AI improve productivity?” is usually too broad a question. Productivity is not an inherent property of a model. It is a result produced by a worker, a task, a tool, a process, and a measure.

In a staggered rollout involving 5,172 customer-support agents, Erik Brynjolfsson, Danielle Li, and Lindsey Raymond found that access to a generative assistant increased issues resolved per hour by 15 per cent on average. The gain was much larger for novice and lower-skilled workers and small for the most experienced workers [6]. That is strong evidence for a specific assisted workflow. It is not evidence that every function becomes 15 per cent more productive, or that the assistant can own the customer relationship.

A randomized study of experienced open-source developers reached a different result. Sixteen developers completed 246 real tasks in repositories they knew well. With early-2025 AI tools, they took 19 per cent longer, even though they believed the tools had made them faster [7]. The sample was small and the result belongs to those developers, repositories, tasks, and tools. It should not become a general claim that coding agents slow engineers.

Nor should it be dismissed because tools improved later. When METR tried to update the estimate in 2026, participation choices, task selection, non-compliance, and concurrent agent work made the treatment effect difficult to identify. The researchers thought newer systems probably offered more acceleration, but described the evidence for the magnitude as weak [8]. The methodological difficulty is itself instructive: once people delegate, supervise several runs, or choose different work because an agent is available, keyboard time stops being an adequate measure.

The support and software studies do not cancel each other. They describe different systems. One offered recommendations inside a high-volume workflow with observable outcomes. The other involved experienced maintainers changing mature codebases where understanding, integration, and review are substantial parts of the work. A person may become faster at a task while the system around that person remains unchanged—or becomes slower under the added load.

DORA’s 2025 research on software delivery describes AI as an amplifier of the surrounding organization: strong systems can convert local acceleration into performance, while weak ones can produce instability faster [9]. That finding is observational, not proof of a universal law. It nevertheless points toward the right unit of analysis. Faster generation alone does not tell us how much organizational value was created.

We need four different words:

Term	Meaning	A misleading proxy
Output	An artifact or action produced: code, analysis, copy, a recommendation, a transaction	Number of artifacts generated
Throughput	Accepted work that crosses the complete operating path	Tasks started or drafts produced
Outcome	A change in customer, operational, or strategic conditions	Work shipped without measured effect

Term	Meaning	A misleading proxy
Value	The benefit of that outcome after cost, risk, maintenance, and foregone alternatives	Model cost or hours reportedly saved

Cheap output is useful. It is not the same as throughput, outcome, or value. The difference matters to a COO because the company pays for the complete system while each team tends to report the part it produced.

1.3 The constraint moves

At any moment, end-to-end performance is limited by one or more constraints. Expanding capacity at one constraint is valuable until another limits the result. If a company can generate analyses, designs, code, and communications much faster without changing how it chooses, integrates, checks, and authorizes them, scarcity moves elsewhere.

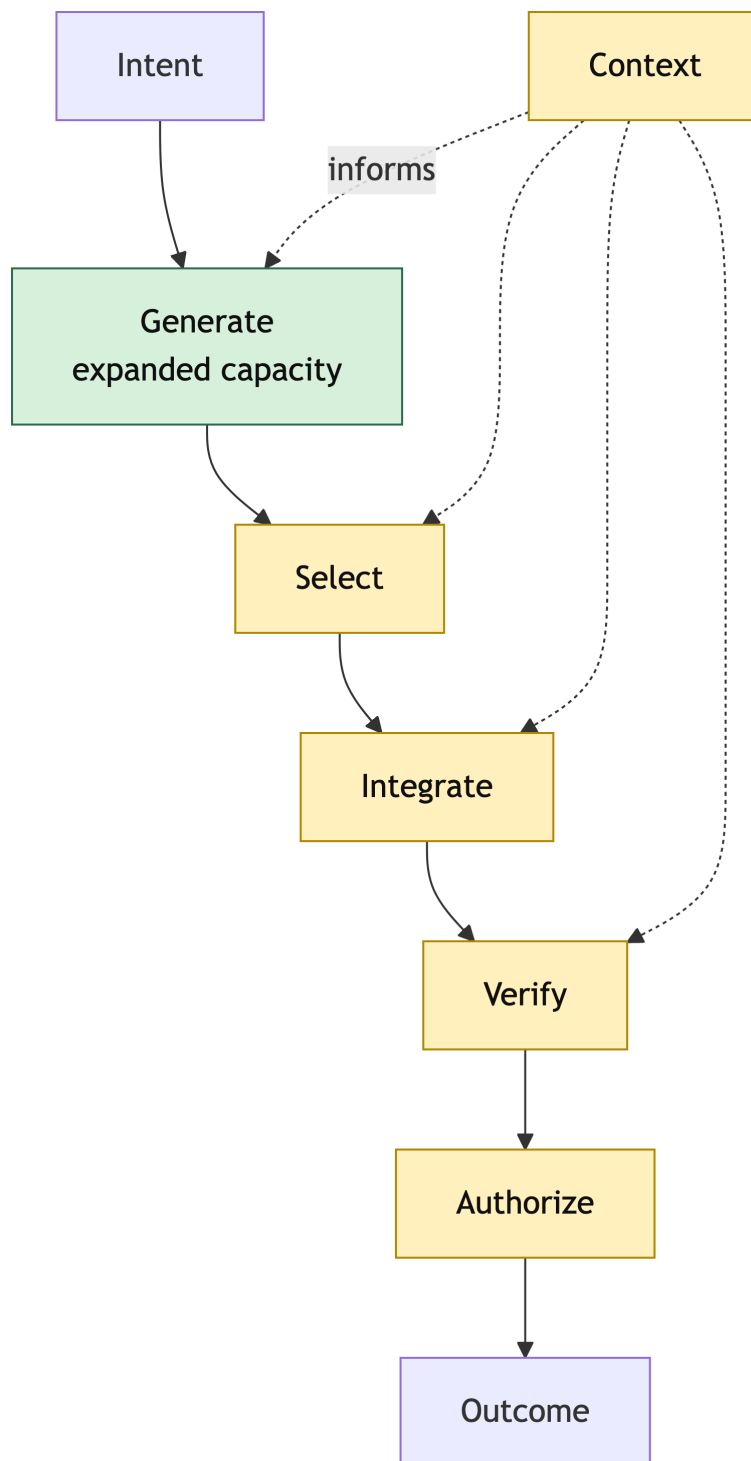


Figure 1.1: When generation capacity expands, the yellow capacities can become queues in the surrounding operating system.

The exact sequence varies. Context can shape generation or arrive later to test an option. Verification may run continuously. Low-risk actions may be authorized in advance. The diagram is not a workflow template. It identifies five capacities that cheap production can expose.

None of these capacities is new. What changes is the imbalance between them. Generative systems can increase the arrival rate of work without increasing the capacity of teams that must absorb it. Agents can

also participate in the downstream steps, changing who or what selects, checks, and acts. The constraint can therefore move faster and become harder to see in conventional activity measures.

Selection. Which request deserves work? Which option deserves commitment? Generation creates alternatives; it does not make the opportunity cost of choosing one disappear. A portfolio of cheap experiments can still consume the scarce attention needed to stop most of them.

Context. Does the work reflect the organization's current intent, facts, constraints, history, and policy? A fluent answer built from stale commercial terms or the wrong customer state is fast rework. Supplying context is not a one-time prompt-writing exercise. Context must be current, accessible, permitted, and traceable.

Integration. Can the output enter a product, process, data environment, or customer relationship without transferring work to someone else? A generated feature that creates a new service dependency is not finished when its code runs. A proposal that cannot survive finance, security, or delivery constraints is not organizational throughput.

Verification. What evidence is proportionate to the consequence? A private brainstorming note and a payment instruction should not share an evaluation regime. Verification can include tests, source checks, simulations, peer review, monitoring, or comparison with a known baseline. It need not mean inspecting everything manually. It should explain why the system is trusted enough to act.

Authorization. Who—or what—may publish, deploy, spend, promise, reject, or bind the company? An agent may be technically capable of taking an action without having legitimate authority to take it. Authorization converts capability into bounded institutional power.

Accountability surrounds all five. It is not another box through which work passes. It answers a different question: who remains answerable for the outcome when work is divided among people, models, software, and vendors?

For the scaling COO, these queues often collapse into one calendar. Selection arrives as portfolio review. Missing context arrives as a request to recall why an earlier promise was made. Integration arrives as conflict between a business unit and a shared function. Verification arrives as a document that needs “one final look.” Authorization arrives as a message asking, “Are you comfortable if we proceed?”

The founder experiences these as unrelated interruptions. They are often one design failure: the organization depends on a person where it needs a decision system.

1.4 Cheap creation has an option cost

The apparent price of an additional option approaches zero long before its total cost does.

Return to the growing company. The fourth plan for the new business line may cost almost nothing to generate. It still asks the leadership team to compare assumptions, resolve conflicts with the existing business, test the economics, decide which capabilities remain shared, and state who may commit the company. Even a rejected plan can leave residue: forecasts, prototypes, provisional hires, technology choices, stakeholder expectations, and exceptions that somebody must later unwind.

A more honest cost equation is:

Total system cost = generation + selection + context + integration + verification + authorization + failure recovery + maintenance.

This is not an accounting standard. It is a checklist for costs that pilots often omit. If a pilot records only license fees and minutes saved, it can report a local gain while exporting cost to reviewers, platform teams, risk owners, or future maintainers.

The opposite mistake is to demand that every use case prove enterprise-wide value before it begins. Learning also has value. A small experiment can be justified by the uncertainty it resolves. But that requires an explicit learning question and a stop decision. “We can generate it” is not a reason to keep it.

1.5 Won't agents absorb the new queues?

They will absorb parts of them. Agents can rank alternatives, retrieve context, open pull requests, run evaluations, and act within pre-approved limits. As capability improves, the constraint will move again.

That does not make operating design temporary. It makes operating design continuous.

An agent selecting investments still needs an objective and evidence of decision quality. An agent integrating code still needs access boundaries, tests, and a recoverable deployment path. An agent approving a refund has been given authority that must have a source, a limit, and a record. More capable execution

changes where humans intervene; it does not remove the need to decide how intent, authority, evidence, and learning move through the company.

The claim also has boundaries. It is strongest in digital, decomposable, repeated work with observable results. Physical constraints may dominate manufacturing or logistics. Trust and relationship may dominate negotiation or care. Ambiguous strategy cannot be made sound merely by generating more strategic options. Stable processes should not be made fluid for the sake of appearing modern.

The point is not that production no longer matters. It is that production can no longer be assumed to be the limiting step.

1.6 What to do on Monday

Choose one recurring decision that still returns to the founder or COO. Use a real decision: a price exception, senior hire, product commitment, capital request, customer promise, or dispute between a business unit and a shared function.

1. Name the decision and the business outcome it is meant to protect. Do not name a meeting or document.
2. Write down the formal owner, then the person who makes the decision in practice.
3. Trace where the decision waits: selection, context, integration, verification, or authorization.
4. Record what the COO knows, checks, or weighs that the stated owner cannot yet access or apply.
5. Design one delegation test for the next occurrence: a clear intent, decision boundary, evidence requirement, and escalation condition. Do not silently retake the decision when the answer differs from your preference.

This exercise will often reveal a paradox: the organization is producing more while becoming harder to steer. Decisions multiply. Dependencies accumulate. Review work arrives without capacity. Exceptions live in messages and meetings. The COO remains indispensable not only because others lack skill, but because the company has stored context, judgment, and authority in one person.

That is not solved by telling the COO to let go. It is solved by making the missing operating structure explicit.

That liability is **Hidden Operational Debt**. It is where the next chapter begins.

2. Hidden Operational Debt

Most companies already have the evidence of how they really operate. It sits in customer conversations, CRM histories, issue trackers, approval threads, incident logs, calendars, financial systems, and the software itself.

Management rarely sees that evidence as a connected record. It receives a monthly summary. The summary says revenue is on plan, delivery is broadly on time, and customer satisfaction is stable. It does not show that pricing decisions now cross four desks, that one product manager resolves half the conflicts between two teams, or that the same customer problem has returned under six different labels.

Then the COO asks an AI assistant what is going wrong. The assistant reads a handful of documents and produces a familiar verdict: clarify ownership, improve communication, streamline processes.

The verdict is not false. It is almost useless.

A manager no longer needs a chapter to learn that organizations accumulate debt. Nor does a manager need six new labels for familiar dysfunction. General-purpose AI can propose a classification of the symptoms in seconds. The useful question is harder: **what evidence would allow the company to see the liability forming, estimate its effect, and intervene before another layer of management is added to absorb it?**

That is where Hidden Operational Debt becomes more than a metaphor.

2.1 The company is leaving a trail

Joshua Gans frames the economic effect of AI primarily through prediction and its contribution to decision efficiency [3]. Inside a company, however, a better prediction has to travel through an operating path. Someone must trust the inputs, understand the tradeoff, possess the right to decide, execute the decision, and observe what happened. A cheaper prediction has little value when that path is broken.

Hidden Operational Debt is the accumulated liability in those broken paths. It appears when the way the company is expected to operate and the way work actually moves have drifted apart.

The monthly report records the eventual output. The operational trail records the struggle:

- when a request entered the system;
- where it waited;
- which information was missing;
- how many people touched it;
- which rule or precedent was used;
- when a human overrode an automated recommendation;
- whether the decision was reversed;
- what the customer experienced;
- and whether the economic result matched the original assumption.

These events used to be expensive to assemble. Much of the trail was unstructured, so diagnosis depended on interviews, workshops, process maps, and the memory of experienced operators. Those methods still have value, especially when the record is incomplete. But they are no longer the only option. Language models can read thousands of tickets or decision threads, recover recurring themes, connect similar exceptions expressed in different words, and draft a first map of where the operating path keeps breaking.

This changes the economics of organizational diagnosis. It does not make the diagnosis automatic. The model can misclassify an exception, miss political context, or confuse correlation with cause. Its advantage is breadth and speed. It gives management a way to inspect actual workflows without commissioning a three-month study.

The source events must remain attached. A plausible summary without traceable evidence is merely a faster opinion.

2.2 Debt has an observable fingerprint

Six forms of debt are useful here, but as search lenses rather than a maturity model. Each leaves a different fingerprint in the data. Each is also affected differently when AI increases the volume and speed of work.

Debt	Signals already present in the business	What AI tends to amplify
Decision	Queue time, repeated approvals, executive escalations, overrides, reversals	More recommendations reach the same unresolved decision boundary
Context	Missing fields, conflicting sources, stale policies, time spent reconstructing history	Fluent answers spread weak or outdated context faster
Coordination	Handoffs, blocked work, re-opened tasks, dependency chasing, recurring alignment meetings	More work starts in parallel and arrives at shared dependencies sooner
Integration	Manual re-entry, reconciliation, failed syncs, bespoke handoffs, local data copies	Agents scale brittle interfaces and produce workarounds faster than they are retired
Knowledge	Repeated questions, rediscovered failures, undocumented rationale, inconsistent onboarding	Generated content grows while reusable judgment remains scarce
Control	Permission exceptions, missing evidence, after-the-fact approvals, unowned incidents	Action volume and access expand faster than review capacity

This is not a scoring system. One broken path can leave several fingerprints. A contract that waits five days may involve missing margin data, unclear discount authority, and a manual transfer between the CRM and finance model. Forcing it into one category would make the analysis worse.

The categories matter because the remedies differ. Decision debt is not repaired by a better summary if nobody has authority. Context debt is not repaired by delegation if the delegated owner cannot trust the numbers. Coordination debt is not repaired by another integration when the teams have incompatible priorities.

AI also changes the impact profile. In a manual organization, ambiguity limits throughput: only so much work can reach the unresolved boundary. Once agents can prepare analyses, write code, contact suppliers, or launch campaigns, far more work can arrive there. The debt may look smaller per task while becoming larger in aggregate.

The 2025 DORA research describes AI in software development as an amplifier of the surrounding organizational system rather than an independent performance intervention [9]. For operating-model design, the working hypothesis is that the same mechanism travels beyond software. A clear operating path becomes faster. A confused one produces confusion at machine speed.

2.3 Real practice: agent builders trace the path

The clearest public operating practices come from teams building agent products. They cannot manage a probabilistic system by reading its final answer alone. They retain the trace: the input, retrieved context, intermediate actions, tool calls, handoffs, errors, human feedback, and final state.

Anthropic's guidance makes a useful distinction between a transcript and an outcome. An agent can state that it booked a flight; the outcome is whether a valid reservation exists in the database. Its evaluation practice combines traces with outcome checks and tracks measures such as latency, token use, cost per task, and error rate. It also recommends sourcing new tests from failures seen in real use rather than relying only on synthetic examples [10].

That distinction belongs in the operating model of the company. A team can report that a customer issue was resolved; the outcome may be a repeat contact two days later. A business-unit lead can report

that a pricing decision was delegated; the trace may show that the COO rewrote the recommendation in a private message. A product team can close a feature request; the outcome may be lower activation because the underlying customer problem was misunderstood.

Customer-service systems show the same pattern in a narrower domain. Intercom’s current reporting for its Fin agent does not stop at how often the agent was used. It separates involvement, resolution, escalation, customer experience, and the performance of the content sources used to answer customers. The underlying conversation record includes the outcome state, rating, escalation reason, and content sources [11]. Those measures make it possible to ask whether automation produced a result, where it handed work back to a person, and which knowledge failed.

Agent observability tools take the approach further. Production conversations can be grouped into recurring categories while retaining links to the underlying traces. Error rate, latency, cost, evaluator feedback, and extracted attributes can then be compared by category and over time [12]. The point is not to endorse a particular tool. It is to notice the operating discipline: **keep the path, connect it to the outcome, and make recurring exceptions visible**.

Companies can apply the same discipline outside an AI product. A pricing approval, supplier exception, customer onboarding, incident response, or new-business launch is also a trace. It has a trigger, context, steps, decisions, exceptions, actors, and an outcome. Once that record is available, AI can help interpret it continuously.

2.4 The dashboard should show strain, not activity

Most executive dashboards describe results after the operating system has absorbed the cost. Revenue arrived. Tickets closed. Features shipped. Hiring completed. The company may be hitting those numbers through escalation, rework, and executive intervention that will not survive another doubling in volume.

An operational-debt dashboard sits beneath the result. It should let a leader move through three views.

The first is the **outcome**. Did the customer’s problem stay resolved? Did the deal produce the expected margin? Did the release improve the intended behavior? Did the new business line consume the shared capacity assumed in its model? A proxy such as “task completed” is not enough when the real result can be checked.

The second is the **path**. How long did the work wait compared with the time spent doing it? How many handoffs occurred? How often was an item reopened, reversed, or escalated? Where did the path diverge from policy? Which step repeatedly lacked context?

The third is the **system boundary**. What did people decide, what did agents execute, and where did control move between them? Useful measures include autonomous completion without correction, human intervention, policy exceptions, failed tool calls, missing evidence, and cost per successful outcome. Adoption is not a result. A high rate of AI use can coexist with poor economics and more rework.

No universal set of thresholds exists. A safety-critical approval and a low-value customer refund should not have the same path. Medians can also hide the cases that consume executive attention, so distributions matter. A COO often learns more from the slowest ten percent of consequential decisions than from average decision time.

The dashboard therefore needs drill-down, not decorative precision. Every aggregate should open into the events behind it. If escalations rise, management should be able to see whether they cluster around one rule, customer type, business unit, data source, or agent action. If the system cannot provide examples, the chart is not ready to govern anything.

2.5 Illustrative example: a new business line seen through its traces

Consider a company moving from one activity to several. The founder-COO launches the second business line personally, then appoints a business-unit lead and hands delivery to a Head of Operations. The organization chart says responsibility moved. Pricing behavior says otherwise.

The usual dashboard shows pipeline, bookings, gross margin, and perhaps the number of deals awaiting approval. It does not explain why pricing keeps returning to the founder.

The company already has most of the required evidence. The CRM holds the proposed price, customer segment, dates, owner, and deal result. Finance holds the cost assumptions. The contract system records non-standard terms. Product and operations systems expose capacity constraints. Email or messaging threads contain the exception rationale and the moment the COO became involved.

An AI analysis can turn the unstructured parts into proposed events: exception reason, missing context, policy cited, people consulted, decision made, and rationale. Those fields should be sampled and corrected

by the operators who understand the process before they become metrics. Once calibrated, the analysis can run across the full period.

The resulting view might show:

- pricing-exception volume and rate by business unit and customer segment;
- median and tail decision time, separated into working time and waiting time;
- number of handoffs and requests for missing information;
- share of exceptions escalated to the COO, with the reasons for escalation;
- concentration of decisions by person, rule, and shared dependency;
- margin at approval compared with realized margin;
- reversals, contract amendments, and repeat exceptions for the same cause;
- deals lost or delayed while an exception remained open.

Now management can distinguish several stories that previously sounded identical. If most delays occur because shared-asset cost is unavailable, transferring pricing authority will not solve the problem. If the data is present but the business-unit lead escalates every deviation from the founder's preference, the liability sits in decision rights or behavior. If an agent proposes discounts inside the agreed envelope but humans repeatedly override them without recording why, the company has created an unobservable control loop.

Alerts should emerge from those stories. The system might flag a new cluster of exception reasons, a material change in tail decision time, repeated use of a stale cost model, an unusual rise in reversals, or growing concentration of approvals around one executive. The threshold must be agreed by the owner and calibrated against the company's own baseline. "Red" should mean that someone knows what to inspect and has the authority to respond.

This dashboard does more than save the COO time. It makes delegation falsifiable. The company can see whether decisions genuinely moved, whether outcomes remained acceptable, and where the next constraint formed.

2.6 Ask AI to build the first version

The first operational-debt view does not require a transformation program. It needs a consequential workflow, enough history to reveal recurrence, and access to an approved AI environment. Sensitive customer, employee, financial, and contractual data should be minimized or anonymized, and only placed in a system the company permits for that information.

Choose one path that is already causing concern: pricing exceptions, customer onboarding, incident resolution, supplier approval, or product requests. Export the last several months of records from the systems involved. Include timestamps and outcomes, not only narrative descriptions. Add the current policy, decision-rights document, or operating rule if one exists.

Then give Claude, ChatGPT, or another capable model a brief like this:

Act as an operating-system analyst. Analyze the attached, anonymized records for one workflow. Reconstruct the common path from trigger to outcome. Identify recurring waiting, handoffs, missing context, escalations, reversals, exceptions, manual reconciliation, and control failures.

For each pattern, separate: (1) what is directly observed in the records, (2) what you infer, and (3) what data is missing. Link every finding to representative source records. Do not rank employees or treat activity volume as performance.

Produce four outputs: a proposed event schema; a ranked set of operational-debt hypotheses; a dashboard specification with metric definition, source, owner, and drill-down; and alert rules calibrated to the observed baseline. Connect path measures to customer, quality, or economic outcomes wherever the data permits. State clearly where no causal conclusion is possible.

The first response will expose data gaps and bad definitions. That is useful. Ask the model to generate the transformation logic, queries, or formulas only after the operating owner confirms that the proposed events mean what the model thinks they mean.

Then make the build request explicit:

Using only the validated fields, generate a working local dashboard or an implementation specification for our existing BI tool. Create three views: outcome, operating path, and human-agent boundary. For every metric, show the formula, data source, refresh cadence, owner, and links to the

underlying records. Add alert logic for changes against our baseline. Do not invent unavailable fields; list them as instrumentation gaps.

The exact output may be an HTML prototype, SQL and chart definitions, or a specification for Power BI, Looker, Metabase, or the company's existing tool. There is no reason to begin with a new platform.

Before relying on the result, take a sample from each major pattern and compare the AI classification with the source material. Keep the corrections. They become examples for the next run and the beginning of an evaluation set. When a new failure appears, add it. This is the same feedback loop used by teams that improve production agents: real traces become tests, tests protect the definition, and the definition makes the dashboard more trustworthy.

The output should not be a one-time diagnostic deck. Schedule the analysis at a cadence appropriate to the workflow. Let the system surface new clusters and threshold breaches. Let the owner inspect the underlying cases, decide whether a liability is real, and record the intervention. Then watch whether the path and outcome change.

That final step matters. A dashboard that detects recurring debt but cannot trigger a decision becomes another source of operational debt.

2.7 From reporting to an operating instrument

Operational visibility can easily become surveillance. Measuring handoffs and delays at individual level invites gaming and punishes the people who absorb the hardest exceptions. The unit of analysis should normally be the workflow, decision boundary, policy, interface, or recurring condition. Individuals appear when they own the remedy, not as targets on a leaderboard.

AI-generated categories can also harden a weak interpretation. Preserve samples, let operators challenge classifications, and show confidence and missing data. When financial or customer consequences are uncertain, say so. The aim is not a magical debt score. It is a shorter path from signal to investigation to operating change.

Once that path exists, Hidden Operational Debt stops being an annual workshop topic. It becomes observable state. Customer conversations reveal where policy and product diverge. Software logs reveal where work repeatedly fails or returns. Decision traces reveal where delegation is nominal. Agent traces reveal where autonomy exceeds control or where humans are still paying invisible interest.

The company can then ask a better question than "Where should we use AI?"

It can ask: **Where is our operating system preventing intelligence - human or artificial - from becoming a sound decision and a verified result?**

That question leads to the boundary between adding an AI tool and changing how the company operates. It is the subject of the next chapter.

3. Why Tools Do Not Change the Operating Model

The easiest AI pilot to approve is attached to a task one person already owns. Draft the email. Summarize the meeting. Produce the first analysis. Write the test. The user can adopt the tool without asking anybody else to change.

That is why early gains appear first in individual work. It is also why they can mislead management about the company.

A randomized field experiment gave half of 6,000 knowledge workers access to generative AI inside their normal office applications. Workers who used the tool spent less time on email and appeared to complete documents faster. Meeting time did not change significantly. The authors' explanation is revealing: behavior that one person could change moved before behavior requiring coordination with others [13].

The tool improved a local action. The shared routine remained intact.

3.1 What an operating model contains

An operating model is the set of arrangements that converts strategy into repeated results. It includes more than process. It determines:

- what the company is trying to achieve and what takes priority;
- what information is authoritative;
- where decisions sit and how they escalate;
- how work crosses teams, systems, and business units;
- which controls bind action;
- what evidence is retained;
- and how results change the next decision.

An AI tool changes the operating model only when it changes one or more of those arrangements. A meeting summary may save time. A decision record that automatically captures the evidence, owner, authority, dissent, and later outcome changes how the company learns. The interface looks similar; the institutional effect is different.

Consider a discount approval. Sales prepares the request, finance checks margin, legal checks terms, and the COO approves. An assistant can draft each handoff. The request moves through the same rights, queues, and data gaps, only with better prose. If the company instead defines a discount envelope, makes current cost and capacity visible, delegates decisions inside the envelope, and logs exceptions with outcomes, the operating model has changed. AI may support the new path. It did not cause the redesign.

3.2 The paved cow path

Automation often preserves an old compromise because the current process is the easiest specification available. The team maps the steps, connects the systems, and asks an agent to move work between them. Nobody asks why the steps exist.

This is the paved cow path failure. A temporary approval becomes a permanent API. A spreadsheet built to reconcile two unreliable systems becomes an automated reconciliation service. A weekly status meeting becomes an AI-generated status pack. Friction declines, so the underlying liability attracts less attention.

The result can be rational. Paying the interest more cheaply may be worthwhile while the company prepares a deeper change. The error is reporting the saving as transformation.

There are three levels of improvement:

Level	What changes	Typical result
Task	One person produces an artifact faster	Local time saving
Workflow	Several steps, handoffs, or systems are redesigned	End-to-end throughput or quality improves
Operating model	Intent, rights, interfaces, controls, evidence, or learning change	The company can behave differently at scale

The levels build on one another. A task improvement can fund a workflow redesign. A successful workflow can reveal a better allocation of authority. Problems begin when management assumes the levels are interchangeable.

3.3 Local output can create system load

Every team can optimize itself and make the company slower.

Marketing generates more campaigns, increasing review and sales follow-up. Product produces more specifications, increasing engineering selection work. Coding agents create more pull requests, increasing integration and security review. Finance generates more scenarios, increasing the number of choices leadership must resolve. The local dashboard is green because output rose. The system queue grows elsewhere.

The correct unit of analysis is the complete path from intent to verified result. Ask where work waits, where it returns, where a human reconstructs context, and where exceptions concentrate. If the gain disappears when downstream review, rework, failure recovery, and maintenance are included, the pilot has moved cost rather than removed it.

This does not require a central transformation office to approve every use. Teams should experiment. But every material pilot needs an owner for the end-to-end result, not merely the tool. That owner should know which metric may improve, which downstream cost may rise, and what would justify stopping.

3.4 The test

Before calling an initiative an operating-model change, complete this sentence:

We used to rely on [**old intent, context, right, interface, control, evidence, or learning mechanism**]. We now rely on [**new mechanism**], which allows [**specific changed behavior**] while protecting [**specific consequence**].

“Employees now use copilots” fails the test. “Business-unit leads now decide discounts inside a live margin and capacity envelope, while exceptions route to finance with a trace” passes it.

The distinction protects the company from two mistakes. The first is underestimating a useful local tool because it did not transform the enterprise. The second is overestimating adoption because many people opened an application.

ATOM begins where the tool ends. It describes the arrangements around intelligence: how intent enters, how context is governed, how decisions receive authority, how humans and agents execute, what evidence returns, and how the company learns.

II

Part II: The ATOM Model

4	ATOM: Intent In, Evidence Out	27
4.1	Intent is a usable constraint	27
4.2	Context is governed state	28
4.3	A decision is a commitment	28
4.4	Execution is a designed combination	28
4.5	Evidence is an output	28
4.6	Learning has an owner	28
4.7	What ATOM is not	28
5	Context Is Infrastructure	31
5.1	The context envelope	31
5.2	Separate memory by purpose	31
5.3	Make authority visible	31
5.4	Build context as a product	32
6	Design the Decision Before the Agent	33
6.1	Inventory commitments, not meetings	33
6.2	Consequence and reversibility shape autonomy	33
6.3	Authority is specific	34
6.4	Prediction does not supply judgment	34
6.5	Turn a recurring escalation into a specification	34
7	Human-Agent Execution	35
7.1	Choose the right executor	35
7.2	Design the handoff	35
7.3	The Synthesizer	35
7.4	Multi-agent is an architecture choice	36

4. ATOM: Intent In, Evidence Out

An operating model should be drawable without a slide deck.

ATOM has six connected layers and one control plane. Read them clockwise as a loop:

1. **Intent** states the outcome, priority, boundaries, and time horizon.
2. **Context** supplies the authorized facts, policy, history, and current state.
3. **Decisions** convert intent and context into a committed choice.
4. **Execution** combines people, deterministic software, workflows, and agents.
5. **Evidence** records what happened and what result followed.
6. **Learning** changes the relevant assumption, context, policy, capability, or strategy.

The **control plane** crosses the loop. It provides identity, permissions, limits, approvals, monitoring, intervention, rollback, and accountability.

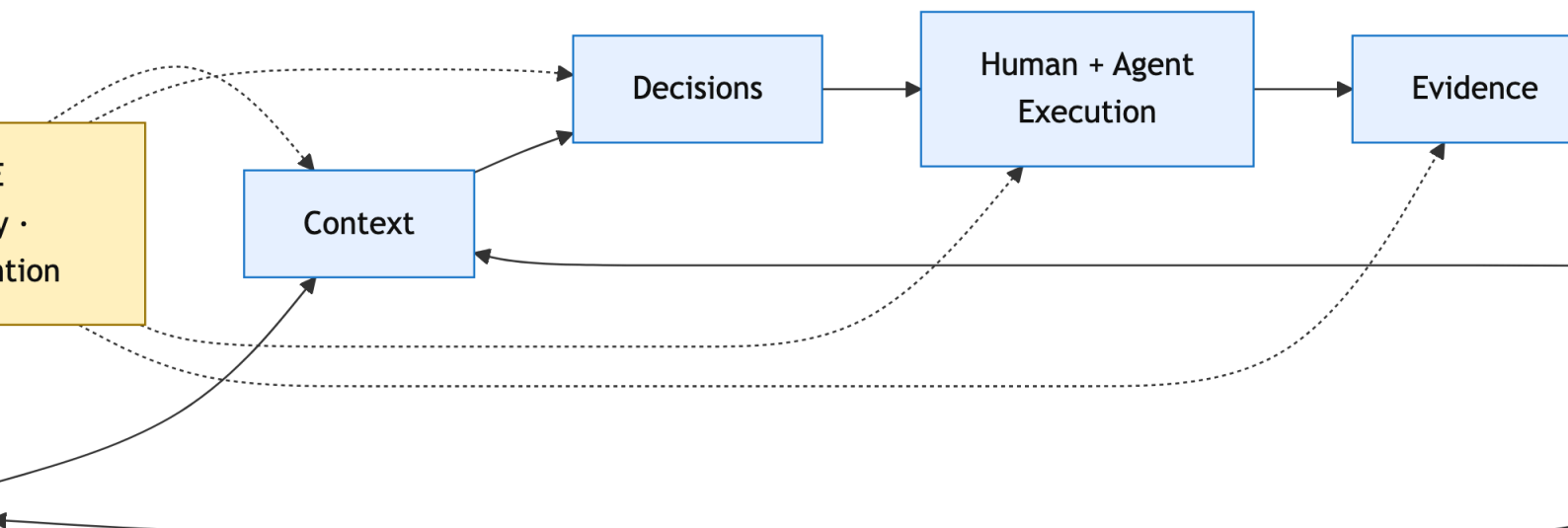


Figure 4.1: The canonical ATOM loop: intent enters, evidence returns, and learning changes the next cycle. The control plane constrains every layer.

That is the complete model. The rest of the book makes each part operational.

4.1 Intent is a usable constraint

“Grow efficiently” is not intent. It does not tell a team what to trade away, which customer matters, or how long the choice should hold.

Usable intent names an outcome and its boundary. A new business unit might be asked to prove that a particular customer problem can support a positive contribution margin without consuming more than a stated amount of shared engineering capacity. The unit now has room to act and a reason to refuse attractive work that violates the experiment.

Intent is not a cascade of slogans. It is a constraint that makes local decisions coherent.

4.2 Context is governed state

Context includes live business data, policy, customer history, commitments, prior decisions, and the state of the work. It also includes provenance: where the information came from, when it was updated, who owns it, and who may see it.

More context is not always better. An agent handling a refund does not need the entire customer data lake. It needs the current account state, the applicable policy, relevant interaction history, and authority limits. Excess context raises cost, creates privacy exposure, and makes conflicts harder to detect.

4.3 A decision is a commitment

Analysis becomes a decision when an authorized actor commits the company. The actor may be a person, a deterministic rule, or an agent acting within delegated bounds. ATOM does not assume the human always makes the final call. It requires the source and limit of authority to be explicit.

Every recurring decision should expose its owner, inputs, criteria, consequence, reversibility, evidence requirement, and escalation path. Without these, an agent may offer advice but cannot receive legitimate autonomy.

4.4 Execution is a designed combination

Execution belongs to the system best suited to the work. Deterministic software handles stable rules. Models interpret ambiguity. Agents manage bounded multi-step work. People set intent, exercise judgment, negotiate conflict, absorb legitimate exceptions, and remain accountable.

The design question is not “human or AI?” It is where each hands work to the other, what state transfers, and what happens when the receiver cannot proceed.

4.5 Evidence is an output

ATOM produces two outputs: a business result and evidence about how the result was produced. Evidence includes source versions, decisions, approvals, actions, tool responses, exceptions, cost, and outcome measures. Its depth should match the consequence.

An organization that only records the final answer cannot distinguish a lucky result from a reliable system. It also cannot investigate harm, improve an agent, or defend the decision later.

4.6 Learning has an owner

Learning is not a generic retrospective. Evidence may suggest that the model needs a new evaluation, the context source needs repair, a permission is too broad, a decision threshold is wrong, or the strategy itself failed. These changes belong to different owners. Letting one agent rewrite all of them would collapse learning and control into the same authority.

Approved and reusable changes enter the **ATOM Playbook**: the maintained collection of decision specifications, unit contracts, context products, controls, evaluation cases, and operating lessons used in future loops. It is curated memory, not a transcript archive.

ATOM closes the loop by sending approved changes to intent or context. It does not let every observation alter production behavior immediately.

4.7 What ATOM is not

ATOM is not an autonomous company. Organizations contain power, conflict, law, identity, trust, and consequences that cannot be reduced to a feedback diagram.

It is not a technology stack. A company can implement the loop with ordinary databases, workflow tools, meetings, and carefully bounded assistants. New infrastructure is justified only when the current tools cannot provide the needed context, action, or evidence.

It is not a replacement for Lean, Agile, product operating models, or sound functional management. It inherits their concern for flow, feedback, customer value, small batches, and capable teams. It adds a more explicit architecture for decision rights, machine actors, evidence, and control.

Nor is ATOM the org chart. The same loop can operate inside a function, across a customer journey, or within a business unit. At scale, repeated loops can form semi-autonomous units, provided their interfaces and central constraints are clear.

The shortest definition is therefore the most useful:

ATOM is a governed learning loop that converts intent and authorized context into decisions, human-agent execution, and evidence.

A reader should now be able to draw it. Start with the six layers. Draw the return from learning. Place control across the path. If an implementation cannot say what enters, who may decide, what may act, what proof returns, and who may change the system, it is not yet an ATOM loop.

5. Context Is Infrastructure

The first failure of an agent is often blamed on the model. In a company, the failure more often begins with the material the model was allowed to see.

Imagine a supplier-selection agent. It compares price, lead time, and defect history and recommends the incumbent. The answer is well reasoned. It is also wrong. The contract repository contains an old agreement, the current sanctions policy lives in a legal memo, and the operations system uses a different supplier identifier. A senior buyer notices the conflict because she remembers the negotiation.

Calling this a hallucination would let the company avoid the real issue. The agent acted on a context product the organization never designed.

5.1 The context envelope

For each decision or task, ATOM defines a context envelope with six parts:

Part	Question
Task state	What is happening now, and what has already happened?
Enterprise facts	Which business data is required, and which source is authoritative?
Policy	Which rules, thresholds, and obligations apply?
History	Which prior decisions, commitments, and outcomes matter?
Provenance	Where did each item come from, who owns it, and how fresh is it?
Access	Who or what may retrieve, combine, retain, or disclose it?

The envelope is deliberately smaller than “company knowledge.” It contains the minimum authorized context needed for the work.

This matters because retrieval is not truth. A search system can return an obsolete policy with perfect relevance. A long conversation can contain a decision that was later reversed. A model can merge two customer records into one fluent account. Context has to be governed before it is generated into prose.

5.2 Separate memory by purpose

Organizations often ask for an agent with memory as if memory were a single feature. It is better to separate four kinds.

Working state lasts for the task: what has been tried, what remains open, and what tool returned which result. **Relationship memory** retains permitted facts about an account or counterparty. **Institutional memory** preserves policies, precedents, decisions, and outcomes. **Learning memory** contains examples used to evaluate or improve the system.

They need different retention, access, correction, and deletion rules. A customer preference may be useful in a relationship record but inappropriate in a training dataset. A failed agent trace may belong in an evaluation set but not in the next customer’s prompt.

5.3 Make authority visible

Every source used for consequential work should answer four questions: Who owns it? When was it last validated? What does it govern? What replaces it?

If two sources conflict, the agent should not silently choose the one with the better wording. It should apply a declared precedence rule or stop. If a source has expired, the system should surface that fact at the point of use. If a human supplies an exception, the exception should have scope and duration rather than becoming an invisible new rule.

The NIST Generative AI Profile treats governance, content provenance, data privacy, testing, and incident handling as lifecycle concerns rather than prompt-level features [14]. That framing is useful for operating leaders. Context quality depends on ownership and maintenance outside the model.

5.4 Build context as a product

A context product has users, an owner, a service level, and quality measures. The pricing context product for a business unit might include current cost, capacity, standard terms, customer segment, outstanding commitments, and the discount policy. Finance may own cost definitions; operations may own capacity; commercial leadership may own the decision threshold. The product joins them without pretending one team owns the whole truth.

Useful measures include source freshness, retrieval success, conflict rate, missing required fields, access denials, and the share of decisions that require manual context reconstruction. These are not model metrics. They measure the organization's ability to supply governed state.

Start with one decision. Ask an approved AI assistant to compare the documents and exports currently used by the people who make it. Request a source map, conflicts, stale items, missing fields, and proposed precedence rules. Then have the source owners validate the result. The output is the first context envelope, not a company-wide knowledge graph.

An agent's autonomy cannot exceed the quality and authority of its context. When the organization treats context as infrastructure, experienced judgment becomes easier to distribute without pretending it can all be written down.

6. Design the Decision Before the Agent

Companies frequently delegate work without delegating the decision. A manager prepares the recommendation, collects the evidence, and coordinates the stakeholders. The founder still chooses. Installing an agent in that arrangement gives the manager faster preparation and the founder more decisions to absorb.

The remedy begins before the agent is selected. Design the decision.

6.1 Inventory commitments, not meetings

A decision inventory records moments when the company commits money, capacity, access, reputation, or a course of action. “Weekly pricing committee” is not a decision. “Approve a discount outside the standard envelope” is.

For each recurring decision, record:

- the outcome it protects;
- the accountable owner;
- required inputs and their sources;
- criteria and prohibited conditions;
- consequence if wrong;
- reversibility and cost of reversal;
- time available;
- authority granted to people, rules, and agents;
- evidence retained;
- escalation and appeal;
- and the date the design itself will be reviewed.

This inventory often exposes phantom delegation. The business-unit lead owns the result but cannot approve price, hire within budget, or commit shared capacity. Accountability moved on the org chart; authority stayed at the center.

6.2 Consequence and reversibility shape autonomy

Not all uncertainty deserves a human approval. A low-cost, reversible action can often be delegated with monitoring. A consequential but reversible action may be executed within a budget and reported immediately. An irreversible or rights-affecting decision should usually receive stronger review, separation of duties, and recourse.

Consequence	Reversibility	Default design
Low	Easy	Act within policy; log the result
Material	Easy	Act within limits; monitor and sample
Low	Difficult	Propose or require approval
High	Difficult	Named human decision, strong evidence, independent control

This is a starting posture, not law. A high-frequency low-value action can create material aggregate exposure. A reversible public message may still damage trust. The owner has to consider scale, affected parties, and failure concentration.

OpenAI’s public agent-building guidance similarly recommends human intervention when failure thresholds are exceeded and for sensitive, irreversible, or high-stakes actions [15]. The principle is wider than any vendor: intervention should be triggered by consequence and uncertainty, not by a vague demand to keep a human “in the loop.”

6.3 Authority is specific

“The agent can handle refunds” is not an authority grant. A usable grant might allow the customer-service agent to issue a refund up to a stated amount when the account, purchase, and policy conditions are verified; prohibit action when fraud indicators or vulnerable-customer flags are present; require a reason and source references; and route exceptions to a named queue.

The same precision applies to humans. A business-unit leader may set price inside an agreed margin and risk envelope. Finance owns the cost model, not each price decision. The center owns changes to the envelope, not every exception inside it.

True delegation allows a sound decision the founder would not personally have made. If the founder routinely reopens choices that satisfy the agreed criteria, the authority grant is fictional.

6.4 Prediction does not supply judgment

AI can estimate churn, demand, fraud, delay, or the likely result of an option. The company still has to assign value to consequences. Should margin be sacrificed to enter a strategic account? How much false-positive fraud blocking is acceptable? Which customer harm requires recourse even if expected revenue rises?

These are judgment questions. They can be informed by data and challenged by a model, but they require legitimate ownership. Gans’s separation of prediction from the rest of the decision is useful here: cheaper prediction raises the value of complementary judgment and action [3].

6.5 Turn a recurring escalation into a specification

Take the last ten decisions of one type that returned to the COO. Give an approved model the records and ask it to identify repeated inputs, criteria, missing context, reasons for escalation, reversals, and outcomes. Ask it to draft a decision specification and mark every inferred rule.

The owner then does the difficult work: chooses the tradeoffs, names the authority boundary, and decides what evidence is enough. Test the specification against past cases, including the awkward ones. If two competent leaders apply it differently, the ambiguity is still in the decision design.

Only then decide whether the next occurrence should be handled by a person, a deterministic rule, an agent, or a combination. Autonomy is not a property of the model. It is institutional authority granted for a particular decision under particular conditions.

7. Human-Agent Execution

The phrase “human in the loop” hides more than it explains. Which human? At what point? With what information? Able to change what? Accountable for which consequence?

Good execution design begins with the work, not a general rule that humans should approve or agents should automate.

7.1 Choose the right executor

Stable, explicit rules belong in deterministic software. Language models are useful when the work requires interpreting unstructured material or producing variable outputs. Agents become relevant when a model must select tools and manage several steps toward a goal. People remain essential where intent is contested, tacit context matters, consequences require legitimate judgment, or relationships carry the work.

An agent is not simply a better chatbot. It can decide which action to take next, invoke tools, observe the result, and continue. That flexibility is valuable precisely where a fully specified workflow is difficult. It is also the source of risk. OpenAI’s practical definition emphasizes independent task completion through models, tools, and instructions, with the ability to stop and transfer control [15].

The organization should use the least autonomous mechanism that solves the problem reliably.

Level	System behavior	Human role
Assist	Retrieve, summarize, draft	Decide and act
Propose	Recommend an action with evidence	Approve, change, or reject
Act with approval	Prepare the exact transaction	Authorize execution
Act and report	Execute within a narrow policy	Review samples and exceptions
Act within a budgeted policy	Plan and execute several steps inside limits	Own outcome, monitor, intervene

Autonomy can differ by action inside one task. A vendor-research agent may search public sources, compare offers, and schedule interviews. It may draft a purchase request. It cannot accept terms, expose confidential data, or send payment unless those separate authorities are granted.

7.2 Design the handoff

The most expensive failures occur at boundaries. An agent escalates without the work it performed. A human approves without seeing the source. Two agents pass summaries until the original customer constraint disappears. A reviewer becomes a rubber stamp because rejecting the action takes longer than accepting it.

A handoff contract states the trigger, receiver, state transferred, evidence required, time expectation, and return path. “Ask a human if uncertain” is not a contract. “Route to the duty manager when the requested refund exceeds the envelope or identity cannot be verified; attach the customer record, policy version, actions attempted, reason for stop, and proposed next action” is.

Review capacity must be designed with execution capacity. If coding agents can open twenty pull requests while qualified maintainers can review four, the system should limit starts, improve tests, or narrow agent scope. Adding more generation creates a queue, not leverage.

7.3 The Synthesizer

ATOM retains one distinctive role from the original model: the **Synthesizer**. This is not an AI whisperer or meeting facilitator. The Synthesizer maintains coherence across a consequential loop.

The role connects intent to the current operating state, exposes conflicts among functions, makes missing decisions visible, and ensures that evidence reaches the owner who can change the system. In a small company the COO may act as Synthesizer. In a business unit it may be the general manager, chief of staff, product-operations lead, or another operator with enough range and trust.

The role should not become a new dependency. A good Synthesizer improves interfaces and decision design so less work requires synthesis by one person. If every cross-functional decision waits for the Synthesizer, the company has renamed its bottleneck.

7.4 Multi-agent is an architecture choice

Several agents can explore in parallel or divide specialized work. They also create more handoffs, duplicated effort, conflicting state, and difficult attribution. Use multiple agents when the tasks are genuinely separable or independent comparison adds value. Do not use them to imitate an org chart.

Anthropic's evaluation guidance stresses that agent behavior must be judged through both trajectory and outcome, using code-based, model-based, and human graders where each fits [10]. That applies to execution design: test not only whether the final artifact looks good, but whether the system used permitted tools, preserved state, stopped correctly, and produced the intended business result.

The design target is not minimum human involvement. It is minimum ambiguity at the boundary. Humans and agents should each know what they received, what they may do, what proof they owe, and where responsibility remains.

III

Part III: How the System Runs

8	A Week Inside an Adaptive Company	39
8.1	Monday: a signal becomes work	39
8.2	Tuesday: judgment enters the loop	39
8.3	Wednesday: execution meets a boundary	39
8.4	Thursday: the company refuses its own recommendation	40
8.5	Friday: evidence changes the next cycle	40
8.6	What actually changed	40
9	Governance in the Work	43
9.1	The control plane	43
9.2	Permissions are not prompts	43
9.3	Guardrails should fail safely	43
9.4	Treat external content as untrusted	43
9.5	Compliance is a floor	44
10	Observe, Prove, Learn	45
10.1	Record the path and the state	45
10.2	Evaluate before and during use	45
10.3	Do not confuse override with error	45
10.4	Learning requires change authority	45
10.5	Turn incidents into reusable tests	46

8. A Week Inside an Adaptive Company

The following week is an **illustrative composite**. The company does not exist. Every technical action described is feasible with 2026 tools, but the quality of the result depends on data, integration, and control that many companies do not yet have.

The company has 110 people and two business units sharing a product platform, finance, and operations capacity. Anya leads the newer unit. Malik is its Synthesizer. Customer demand is rising, but contribution margin is below the threshold agreed with the executive team.

8.1 Monday: a signal becomes work

The weekly operating view flags a cluster of customer conversations. Mid-market customers want a service configuration that sales has handled through manual exceptions. Support volume is rising around onboarding, and two recent deals consumed far more implementation capacity than forecast.

An analysis agent groups the conversations, links them to CRM records and delivery time, and proposes three explanations. It labels each as inference and attaches the source cases. One explanation looks commercially attractive: standardize the exception as a new offer.

Anya does not open a project. She opens a decision: should the unit test a standardized offer for this segment within the current quarter?

The intent is already explicit. The unit may test growth options that can reach its contribution-margin threshold without consuming more than the capacity reserved by the platform and operations teams. Malik asks the context system for the current cost model, customer evidence, capacity, prior experiments, and commitments made in active contracts.

One source is stale. The implementation cost model has not been updated since a supplier change. The control plane marks the field as expired. The agent can continue exploring, but it cannot recommend a launch.

That stop is not a failure. It prevents speed from converting weak context into a commitment.

8.2 Tuesday: judgment enters the loop

Finance updates the cost assumptions. The analysis changes: the offer remains attractive only if onboarding work falls below a specific range. The agent prepares two test designs and a third option that would outsource part of delivery.

The third option violates a current data-processing constraint. It is removed automatically, with the policy reference preserved in the trace.

Anya and the operations lead review the remaining choices. The operations lead challenges the agent's forecast because the proposed customer mix differs from the historical sample. The model cannot resolve the disagreement. Anya narrows the test to one segment and accepts a slower learning rate in exchange for a cleaner result.

The decision is hers. It is reversible, inside the unit's experiment budget, and within the shared-capacity envelope. The COO is notified but not asked to approve.

8.3 Wednesday: execution meets a boundary

A workflow agent creates the experiment record, drafts the offer, opens product and legal tasks, and prepares the CRM configuration. Deterministic checks validate required fields and price limits. A product agent proposes the smallest software change and opens a pull request with tests.

The pull request fails an integration test because the new configuration changes a shared billing event. The agent attempts one repair, then stops at its retry limit. The platform engineer receives the trace, failed test, changed files, and proposed cause.

The engineer finds that the event contract is ambiguous. Fixing only the code would leave both business units vulnerable to future changes. She brings the interface decision to the platform owner. They define the contract, update the test, and record which fields each unit may extend.

The experiment loses a day. It also repays a piece of integration debt that would otherwise have returned.

8.4 Thursday: the company refuses its own recommendation

The offer is ready for a limited release. The agent recommends ten customers based on fit and expected margin. One is a strategic account already involved in a sensitive renewal. That relationship state exists in the CRM but was outside the original context envelope.

Sales rejects the selection. The rejection is not treated as human resistance or model failure. It reveals missing context. The relationship flag is added to the envelope, and the test proceeds with another customer.

The change is reviewed by the context owner before it enters production. The analysis agent cannot rewrite its own context policy simply because an operator disagreed with it.

8.5 Friday: evidence changes the next cycle

The operating review lasts forty minutes. It is not a recital of completed tasks. The group sees the decision, exceptions, agent actions, human interventions, elapsed time, cost, and current customer outcome in one trace.

Three changes are approved:

- finance owns a freshness target for the implementation cost model;
- the platform event contract becomes an invariant for both units;
- strategic-account status enters the offer-selection context with restricted access.

One proposed change is rejected. Malik suggests increasing the agent's retry limit because the human repair was successful. The platform owner points out that the repair required an architectural decision, not another attempt. More autonomy would have hidden the ambiguity.

The Playbook records the updated context envelope, interface contract, and decision rationale. The customer cases become part of the experiment's evaluation set. The strategy is unchanged; the system around it is better specified.

8.6 What actually changed

The week was not frictionless. Work stopped twice. A human contradicted the model. A launch slipped. Those events are the substance of the operating model, not imperfections to edit out of the story.

ATOM appeared at each boundary:

Layer	Event during the week
Intent	Margin and shared-capacity constraints bounded the experiment
Context	A stale cost model stopped recommendation; relationship state was added later
Decision	Anya narrowed and authorized a reversible test
Execution	Agents prepared work; deterministic checks and people handled their parts
Evidence	Sources, failure, rejection, tool actions, and outcomes stayed attached
Learning	Different owners updated cost, interface, and context rules
Control plane	Policy blocked an option, retry limits stopped an agent, access remained scoped

The COO's role also changed. The old company would have sent the decision upward because it crossed finance, product, sales, and operations. The adaptive company had already designed the unit's authority

and the shared constraints. The COO could inspect the decision without becoming its missing operating system.

That is the practical promise of ATOM. Not fewer humans. Not uninterrupted automation. A company that can encounter uncertainty, stop for the right reason, act with legitimate authority, and retain enough evidence to improve the next cycle.

9. Governance in the Work

Governance fails when it arrives as a committee after the action or as a policy document no system can enforce.

For human-agent work, governance belongs in the path. It determines what context can be retrieved, which tools can be called, what value can be committed, when approval is required, what evidence must be retained, and who can stop the system.

9.1 The control plane

ATOM's control plane has seven operating functions:

1. **Identity:** Every person, agent, service, and vendor connection has a distinguishable identity.
2. **Authority:** Rights are granted for specific resources and actions.
3. **Policy:** Machine-enforceable rules sit beside human-readable intent.
4. **Approval:** Consequential actions route to a qualified person or separated role.
5. **Observation:** The company records actions, denied attempts, exceptions, and outcomes.
6. **Intervention:** Authorized operators can pause, narrow, override, or roll back.
7. **Recourse:** Affected people can challenge or repair a result.

These functions should be shared where consistency matters, but outcome ownership remains in the business. Security may define credential standards. Finance may define payment controls. Legal may define retention. The business owner still answers for whether the agent should be doing the work.

9.2 Permissions are not prompts

An instruction such as “do not make payments above €5,000” is not a permission boundary if the agent holds credentials that allow any payment. A model can misunderstand, be manipulated by retrieved content, or call the wrong tool. The payment system must enforce the limit.

This is ordinary security applied to a new actor: least privilege, short-lived credentials, separation of duties, scoped resources, and revocation. Integration protocols do not remove the need. The Model Context Protocol's authorization specification requires audience-bound tokens and explicitly prohibits token passthrough because otherwise a credential intended for one service can be misused elsewhere [16].

The operating implication is direct. Register the agent as an actor. Do not lend it a founder's account. Grant only the tool, data, action, amount, and duration required for the task.

9.3 Guardrails should fail safely

Some guardrails are deterministic: amount limits, allowed domains, required fields, data classifications, or prohibited actions. Others use models to detect relevance, manipulation, or policy conflict. The two should not be confused. Model-based checks add coverage but also uncertainty.

Layer controls according to consequence. A purchasing agent might draft a request, verify the supplier against approved data, and submit it to the procurement system. It cannot approve the request or release payment. The approver sees the supplier, amount, policy checks, source documents, conflicts, and agent actions. If evidence is missing, the transaction cannot advance.

For reversible low-value purchases, the company may later pre-authorize action inside a budget. It should expand the authority because observed evidence supports the change, not because the agent sounds more confident.

9.4 Treat external content as untrusted

Agents read documents, websites, messages, and tool descriptions that can contain instructions. An attacker may try to make the system reveal data or take an unintended action. A less dramatic failure occurs when a supplier document merely contains language that the agent mistakes for company policy.

Separate data from instructions. Limit which sources can influence action. Validate tool parameters outside the model. Require confirmation when the destination, scope, or consequence changes. Keep sensitive tools unavailable during open-ended research.

The phrase “confused deputy” describes a system tricked into using its legitimate authority for someone else’s purpose. It is not an exotic edge case once agents can cross systems. The control plane should assume that content and tools can be wrong even when the model is behaving as designed.

9.5 Compliance is a floor

The EU AI Act now applies in stages and uses different obligations for different roles and risk categories. Since August 2026, transparency requirements apply to specified uses, while deadlines for high-risk systems extend further under the amended timeline [17]. For high-risk deployments, the regulation includes requirements around human oversight, logging, documentation, data quality, robustness, and deployer responsibilities [18].

This book cannot determine the legal classification of a particular system. The operating model should make compliance possible: named roles, traceable versions, retained logs, competent oversight, and an inventory of where AI affects people. Waiting for a lawyer to reconstruct that inventory after deployment is poor governance.

Governance creates useful speed when routine actions can proceed because limits are real. A control plane is successful when teams know where they are free to act, exceptions reach the right owner with evidence, and dangerous actions cannot bypass the company simply by moving faster.

10. Observe, Prove, Learn

An agent completes a task, the dashboard records success, and the customer leaves.

The tool returned no error. The workflow closed. The agent's answer was fluent. None of those facts proves that the customer received the right result.

ATOM treats evidence as a product of execution. The system must preserve enough of the path to answer three questions: What happened? Did it work? What is allowed to change because of what we learned?

10.1 Record the path and the state

A useful trace connects:

- the triggering request and relevant context versions;
- the model, instructions, tools, and permissions used;
- decisions, tool calls, approvals, denials, and handoffs;
- intermediate and final outputs;
- errors, retries, overrides, and interventions;
- the resulting state in the business system;
- and later customer, operational, or economic outcomes.

The final state matters. Anthropic's distinction is exact: an agent saying a flight was booked is not the same as a reservation existing [10]. In company operations, "invoice corrected" should connect to the ledger entry; "customer retained" should connect to subsequent behavior; "decision delegated" should connect to who actually made it.

Evidence depth should follow consequence. A brainstorming assistant needs little retention. An agent changing production code, employment status, a customer contract, or a financial record needs durable identity, version, action, and approval evidence.

10.2 Evaluate before and during use

Offline evaluation tests the system against a controlled set of tasks before release. Online evaluation observes sampled production behavior, outcomes, and new failure patterns. Neither is sufficient alone.

An evaluation suite should include ordinary cases, boundary cases, known failures, adversarial inputs, and cases where the correct behavior is to stop. Different graders answer different questions. Code can verify a database state or prohibited tool call. A model can assess open-ended quality against a rubric. Domain experts calibrate judgment and review cases where consequences matter.

Pass rate alone can hide the failure that matters. Segment by customer type, action, risk level, language, data source, and version. Track tail latency, cost per successful outcome, human intervention, reversal, and evidence completeness. When an agent changes, compare it with the current system on the same cases before expanding authority.

10.3 Do not confuse override with error

A high human-override rate may indicate a poor model. It may also indicate a cautious deployment where humans correctly handle rare exceptions. A low override rate may mean excellent performance or rubber-stamp review.

The trace needs a reason and an outcome. Did the person possess context unavailable to the agent? Did the policy boundary work? Was the override itself wrong? Was the action later reversed? Without those links, acceptance and override rates become management theater.

10.4 Learning requires change authority

Evidence can lead to five different changes:

Finding	Possible change	Owner
The model fails a known task	Model, prompt, or evaluation	Product/agent owner
Required facts are stale	Context source or freshness rule	Data/context owner
A tool behaves unpredictably	Integration or tool contract	Platform owner
Valid actions violate intent	Policy or authority envelope	Business/control owner
Results are consistently poor	Decision design or strategy	Accountable executive

These authorities should remain separate. The agent that detects a weak policy should not silently rewrite the policy and expand its own scope. A controlled proposal, review, test, and release path protects the organization from self-reinforcing error.

10.5 Turn incidents into reusable tests

After an incident, preserve the trace and the real outcome. Reproduce the condition in a safe environment. Add the case to the evaluation suite. Decide whether the correction belongs in context, policy, tool behavior, or model instructions. Then run regression tests so the fix does not break a previously reliable behavior.

This is institutional learning with a memory. It is stronger than a retrospective action item because the failure can be tested against future versions.

The operating review should therefore examine a small set of signals linked to source evidence: business outcome, path health, agent behavior, control events, and new failure clusters. The question is not whether the dashboard is green. It is whether the company can reconstruct the result, explain why it was acceptable, and change the correct part of the system.

IV

Part IV: Build the First Loop

11	Choose a Decision, Not an AI Project	49
11.1	Score the loop	49
11.2	Compare three candidates	49
11.3	Write the pilot as a decision loop	50
12	Build the Minimum Viable Operating Loop	51
12.1	Days 1-5: establish the baseline	51
12.2	Days 6-10: design authority and context	51
12.3	Days 11-15: build the controlled environment	51
12.4	Days 16-20: run in shadow	51
12.5	Days 21-25: release narrowly	52
12.6	Days 26-30: decide the next authority	52
12.7	Minimum viable ATOM canvas	52
13	Prove Value Before You Expand	53
13.1	Compare complete systems	53
13.2	Read the distribution	53
13.3	Use a four-way gate	53
13.4	Transfer to the run	54

11. Choose a Decision, Not an AI Project

“Deploy an agent in operations” is not a pilot. It names a technology and a department, not a result.

A useful first loop begins with a recurring decision or outcome whose complete path can be observed. The unit might be resolving a service exception, approving a non-standard price, triaging an incident, qualifying a lead, or deciding whether a new business experiment receives another tranche of capital.

11.1 Score the loop

Rate candidates using evidence, not enthusiasm:

Criterion	Question
Value	Does improvement affect a customer, economic, risk, or strategic result?
Frequency	Will enough cases occur to learn within the pilot period?
Latency	Is waiting or rework material?
Evaluability	Can correct behavior and outcome be observed?
Context readiness	Are required facts accessible, current, and permitted?
Reversibility	Can mistakes be detected and repaired?
Consequence	Who can be harmed and how severely?
Integration load	How many brittle systems and teams must change?
Ownership	Is one person accountable for the end-to-end result?
Transfer	If successful, can the loop move into normal operations?

Do not reduce the table to one magic score. A high-value use with no observable outcome is not ready. A technically easy use with no meaningful result is a demo.

11.2 Compare three candidates

An **illustrative scale-up** considers three loops.

The first is an agent that prepares board materials. It is visible and reversible, but low-frequency and difficult to connect to better decisions. The second handles customer-onboarding exceptions. It is frequent, measurable, painful, and linked to activation, but touches sensitive customer data and several systems. The third analyzes funding evidence for experiments in the new business line. It has strategic value and a clear executive sponsor, but only a few decisions will occur during the quarter.

The company selects onboarding exceptions. It narrows scope to one customer segment and one class of exception. The decision is not glamorous. It will reveal context gaps, handoffs, authority, agent limits, customer outcome, and transfer to the run in a few weeks.

The strategic funding loop remains a management redesign candidate, but not the first agent pilot. The board-material assistant can proceed as a local productivity tool without being called transformation.

11.3 Write the pilot as a decision loop

A strong charter fits on one page:

When **[trigger]** occurs, **[owner]** uses **[authorized context]** to make or delegate **[decision]** within **[authority limits]**. Humans, rules, and agents execute **[work]**. The system retains **[evidence]** and measures **[outcome]**. It stops or escalates when **[condition]** occurs. After **[period/case count]**, **[named group]** decides to expand, revise, contain, or stop.

The charter forces a choice about what the pilot is actually proving. Is the uncertainty model capability, context quality, user behavior, integration, economics, or authority? If the team cannot name the learning question, it will interpret any activity as success.

Start where the consequence is bounded but the operating lesson is real. The first unit of transformation is not a department or an AI platform. It is one honest decision loop.

12. Build the Minimum Viable Operating Loop

A demo proves that a model can perform under chosen conditions. A minimum viable operating loop proves that the company can let the system encounter real variation without losing authority or evidence.

The following thirty-day sequence continues the **illustrative onboarding pilot**. It is a sequence of design gates, not a promise that every company should deploy in a month.

12.1 Days 1-5: establish the baseline

The owner selects one customer segment and one exception class. The team exports prior cases and reconstructs the current path: trigger, data sources, handoffs, decisions, waiting, rework, outcome, and later repeat contact. It samples cases with frontline operators because the system record omits work performed in messages.

The baseline includes distribution, not only average: median and tail resolution time, number of touches, escalations, reopen rate, customer activation, and cost to serve. The team writes the decision-loop charter and names a kill condition.

An AI assistant can classify historical cases and propose the event schema. People who perform the work correct the categories. Those corrections become the first evaluation examples.

12.2 Days 6-10: design authority and context

The owner specifies what the agent may do. In this pilot it can gather account state, classify the exception, request missing information, and propose a resolution. It cannot change contractual terms or modify an account without approval.

The context envelope lists required fields, source systems, policy versions, precedence, freshness, and access. Missing or conflicting required context produces a stop, not a guessed answer. Sensitive fields are removed unless necessary.

The team writes handoff contracts for customer clarification, operations review, and policy exception. Each handoff carries the work already performed and the reason the agent stopped.

12.3 Days 11-15: build the controlled environment

The agent receives test credentials with the same shape but less authority than production. Tools validate parameters and return structured results. Logs capture inputs, context versions, calls, outputs, decisions, and errors. Known sensitive actions remain technically unavailable.

The evaluation suite contains routine historical cases, difficult exceptions, contradictory context, missing data, malicious instructions inside customer content, and situations where escalation is correct. Tests check both outcome and trajectory.

The team rehearses failure: revoke access, stop a run, recover from a partial action, and route a customer to a person. If operators cannot see and interrupt the system, the pilot is not ready.

12.4 Days 16-20: run in shadow

The agent processes live cases without acting. Its proposals are compared with normal handling. Reviewers record disagreement reasons rather than a bare accept/reject. The owner watches for systematic gaps: one policy misunderstood, one customer segment poorly represented, one system too stale for the decision.

Shadow mode should not continue indefinitely. It proves readiness for a bounded release, not performance under real responsibility. Reviewers behave differently when nothing can happen.

12.5 Days 21-25: release narrowly

The agent goes live for a limited share of eligible cases. At first it prepares the exact action and a person approves. Cases outside scope are excluded before the model sees them. Retry, time, and cost limits are enforced. An on-call owner can pause the loop.

Daily review focuses on new failure patterns and customer outcome, not a demand for perfect automation. A failure caused by a stale policy goes to the context owner. A failed tool call goes to the platform owner. A sound but unpopular decision goes to the business owner.

12.6 Days 26-30: decide the next authority

The team compares the live cases with the baseline and reads a sample of traces. It includes human review time, exception handling, model and infrastructure cost, customer repair, and maintenance. Frontline operators and affected customers contribute evidence.

The gate has four outcomes: expand authority or volume, revise and retest, contain the loop at its current scope, or stop. Expansion is specific. The agent might gain authority to execute one reversible resolution inside a narrow policy. It does not receive a general promotion.

12.7 Minimum viable ATOM canvas

Before live action, one page should contain:

Element	Required statement
Intent	Outcome, boundary, and learning question
Context	Sources, freshness, provenance, access, missing-data behavior
Decision	Owner, criteria, consequence, reversibility, escalation
Execution	Human, rule, workflow, and agent responsibilities
Evidence	Trace, outcome, evaluation, retention
Learning	Review cadence and owners of possible changes
Control plane	Identity, permissions, approvals, stop, rollback, recourse

The canvas is complete only when the people operating the loop recognize their real work in it. A pilot is ready when its authority and evidence are designed, not when its demo works.

13. Prove Value Before You Expand

The pilot resolved cases faster. Should the company expand it?

Not yet. Faster can mean customers received good outcomes sooner. It can also mean cases were closed earlier and returned later, reviewers absorbed hidden work, or the easiest cases moved to the agent while people inherited a harder queue.

13.1 Compare complete systems

The relevant comparison is the previous operating path against the new one. Include:

- customer or business outcome;
- elapsed and working time across the full path;
- human review, escalation, and repair;
- model, tool, integration, and monitoring cost;
- failure severity and recovery;
- maintenance of prompts, policies, context, and evaluations;
- and effects on the people doing the remaining work.

Do not claim a causal gain from a before-and-after chart when demand, staffing, or case mix also changed. A randomized or phased comparison may be possible. When it is not, match similar cases, show the limitations, and treat the result as operational evidence rather than scientific proof.

13.2 Read the distribution

Average improvement can conceal concentrated harm. Segment results by case type, customer group, channel, risk, and degree of agent involvement. Inspect the tail. A system that improves routine cases but mishandles rare consequential cases may still be valuable, provided those cases are detected and excluded.

Likewise, aggregate labor savings can hide a worse job. If agents absorb simple cases, human operators may face uninterrupted exceptions. Measure queue difficulty, interruption, discretion, and repair burden. The goal is not to preserve every old task; it is to avoid designing a system whose economics depend on invisible human absorption.

13.3 Use a four-way gate

The review should produce one of four decisions:

Decision	Evidence
Expand	Outcomes hold, total economics improve, controls work, and new volume is supportable
Revise	Value is plausible but a named context, design, or reliability gap is repairable
Contain	The loop works only for a stable narrow segment and should remain there
Stop	Outcome, economics, control, or affected-party impact does not justify continuation

Stopping is not an admission that the company “failed at AI.” It proves that the operating model can reject activity that does not create value.

13.4 Transfer to the run

An experiment has not scaled when the founder can operate it. It has scaled when a normal owner can run, monitor, and change it through defined interfaces.

Before transfer, name the business owner, service expectations, support path, incident owner, context owners, evaluation cadence, cost budget, change authority, and retirement condition. Train the people who receive exceptions. Remove temporary credentials and workarounds. Record which assumptions remain unproven.

For a new business line, transfer also requires clear economics and shared-service contracts. If the unit's margin depends on unmeasured founder labor or free platform capacity, the pilot has not proven a scalable business.

The 90-day question is not "Did people use it?" It is "Did this loop improve a result at acceptable total cost and risk, and can the company operate it without exceptional support?" Only a convincing answer earns more authority or scope.



Part V: Scale Without Losing the Plot

14	Fractal Units, Not Fractal Bureaucracy	57
14.1	The unit contract	57
14.2	Invariants and local choices	57
14.3	Illustrative example: two locally correct decisions	57
14.4	Where work belongs	58
15	Allocate Capital, Talent, and Knowledge Dynamically	59
15.1	Fund options, not forecasts	59
15.2	Protect the economics of shared assets	59
15.3	Move expertise without dissolving teams	59
15.4	Treat knowledge as maintained infrastructure	60

14. Fractal Units, Not Fractal Bureaucracy

As a company adds business lines, it faces a familiar choice. Centralize decisions and slow the edges, or decentralize and accept duplication. The choice is false when the organization has not specified what must be shared.

ATOM uses the term **Fractal Unit** for a semi-autonomous group that can run a complete governed outcome loop. “Fractal” does not mean every unit is identical. It means the same operating grammar recurs at different scales: intent, context, decisions, execution, evidence, learning, and control.

The repeatable object is not the team diagram. It is the contract.

14.1 The unit contract

Every unit should be able to state:

Element	Contract
Purpose	Customer or enterprise outcome the unit exists to produce
Economics	Revenue, cost, margin, assets, or risk it owns and can observe
Inputs and outputs	What enters, what leaves, and in what form
Decisions	Rights held locally, shared, and at the center
Dependencies	Services and capacities required from others
Context	Sources and policies required to operate
Controls	Non-negotiable limits and escalation conditions
Evidence	Measures, traces, and review obligations
Learning	What the unit may change and what requires shared approval
Lifecycle	Conditions to grow, split, merge, or close

A unit without outcome ownership is a project team. A unit without visible economics is a cost center pretending to be a business. A unit without bounded rights is a reporting layer.

14.2 Invariants and local choices

Scale depends on making a few things rigid enough that everything else can adapt.

Invariants may include customer identity, financial definitions, security controls, core product interfaces, evidence requirements, and rules for allocating shared assets. Units should not invent their own meaning of active customer or contribution margin. They should not bypass access policy because the central service is slow.

Local choices include workflow, roles, experiments, tools, and decisions inside the unit’s authority envelope. The center should resist standardizing them simply because one unit found a successful pattern. Reuse should follow evidence and compatible conditions, not executive preference.

14.3 Illustrative example: two locally correct decisions

A company adds an energy-services unit beside its original equipment business. Both use the same field engineers and device platform.

The equipment unit schedules engineers to protect installation revenue. The services unit promises response times to win recurring contracts. Each choice is rational inside its own P&L. Together they oversubscribe the same asset, and both units miss commitments.

The first response is a central capacity committee. Every exception moves upward. The meeting becomes the operating system.

The better design exposes available engineering capacity as a shared service with a clear owner, reservation rules, priority classes, cost, and exception path. Each unit decides inside its allocation. The center sets the portfolio tradeoff and capacity buffer. Evidence shows demand, utilization, missed commitments, and the economic value of each use.

The units remain different. The interface becomes stable.

14.4 Where work belongs

Place a capability inside a unit when it requires deep local context, changes frequently with the offer, and can be owned end to end. Share it when scale, scarce expertise, risk consistency, or a common asset creates real advantage. Keep it at the center when the decision sets enterprise intent, allocates capital among units, defines non-negotiable policy, or resolves a conflict the unit cannot legitimately settle.

Do not decide from ideology. Centralization has a latency cost; decentralization has duplication and divergence costs. Make both visible.

The fractal hypothesis has a boundary. Constantly re-forming teams destroys relationships and accumulated context. Physical assets do not reallocate like cloud capacity. Regulated accountability may remain centralized even when execution does not. A stable unit with clear interfaces is often more adaptive than a fluid network that renegotiates ownership every week.

What repeats at scale is a governed outcome loop. When unit contracts are explicit, local autonomy and enterprise coherence stop being opposites.

15. Allocate Capital, Talent, and Knowledge Dynamically

Adaptability is not the frequency with which a company changes priorities. A company that reallocates people every two weeks can be highly reactive and incapable of learning.

Dynamic allocation means that evidence can change the commitment of capital, talent, and shared capacity at a cadence matched to the reversibility of the decision.

15.1 Fund options, not forecasts

An emerging business needs enough capital to resolve uncertainty, not a five-year plan disguised as precision. Release funding in stages tied to evidence: customer behavior, unit margin, asset use, operational feasibility, or a technical capability.

The unit has authority inside the tranche. The portfolio group decides whether evidence justifies the next one. AI can compare assumptions with current signals, surface variance, and simulate alternatives. It should not automatically move capital. Portfolio choices contain strategic judgment and opportunity cost that remain executive responsibilities.

Mature operations need a different cadence. Constantly reopening a factory plan, regulated process, or committed customer program destroys value. “Continuous” strategy means signals remain visible between formal decisions, not that every decision remains permanently unsettled.

15.2 Protect the economics of shared assets

A multi-business company cannot delegate well if units cannot see what they consume. Shared engineering, distribution, data, brand, or physical capacity needs an explicit allocation method. It does not have to reproduce accounting perfection. It must be consistent enough that local decisions reflect real scarcity.

Useful questions include: Which asset constrains growth? What return does each use produce? Which demand is committed versus optional? What buffer protects reliability? Which unit is exporting cost to another?

AI can detect patterns in usage and forecast pressure. The allocation rule still needs an owner and an appeal path. A model optimizing measured return may starve a new activity whose strategic value has not yet appeared in historical data.

15.3 Move expertise without dissolving teams

Talent marketplaces promise to match skills to opportunities. Used carefully, they can make hidden capability visible and help people find work beyond their reporting line. Used badly, they turn employees into interchangeable capacity and managers into bidders.

Team cohesion, domain knowledge, consent, development, and career identity are real constraints. A specialist loan should specify purpose, duration, home team, receiving owner, protected time, learning to return, and what happens when priorities conflict. The employee should be able to challenge an algorithmic match and understand which data informed it.

Illustrative example. The newer business unit needs a reliability engineer for six weeks. An internal matching assistant identifies three people from project histories and declared interests. The highest-ranked candidate is rejected because she is carrying an incident-recovery commitment invisible to the model. The second accepts a time-bounded assignment. Her home team receives backfill capacity, and the platform lesson becomes a shared standard rather than private expertise.

The model accelerated discovery. Human context changed the decision. The contract protected both teams.

15.4 Treat knowledge as maintained infrastructure

Shared knowledge does not improve because the company generates more documents. It improves when reliable decisions, interfaces, policies, and lessons are reusable at the point of work.

Every knowledge product needs an owner, audience, source, freshness expectation, and retirement rule. Search logs and agent failures can reveal unanswered questions. Customer conversations can reveal where policy and reality diverge. Unit reviews can nominate a local pattern for reuse. A shared owner decides whether the pattern becomes an enterprise invariant, an optional template, or remains local.

The ATOM Playbook is this maintained set of operating knowledge. It is not a culture wiki or archive of every meeting. It contains the definitions, unit contracts, decision specifications, context products, controls, evaluation cases, and lessons the company expects others to reuse.

Dynamic allocation is disciplined movement. Capital changes when an option earns another commitment. Talent moves through a contract that respects teams and people. Knowledge travels after it becomes reliable enough to reuse. Adaptability comes from changing deliberately, not keeping everything in motion.

VI

Part VI: Leadership and Control

16	Agent Operations: Control at Scale .	63
17	The Manager After Coordination Gets Cheap	65
18	Accountability Cannot Be Delegated .	67

16. Agent Operations: Control at Scale

The first agent is owned by the team that built it. The fiftieth agent reveals that the company has created a new operating estate.

Agents share models, tools, identities, data, and vendors. They change versions. Their permissions drift. One agent's output becomes another's input. Costs accumulate across teams. A local failure can become a repeated enterprise action.

The company needs an Agent Operations capability. It may begin as a responsibility inside platform or security rather than a new department. Its job is to make the fleet operable without taking business accountability away from the units.

16.1 Register the actors

Every production agent should have a registry entry containing:

- name, purpose, and business owner;
- current version and environment;
- models, tools, data sources, and downstream dependencies;
- identity, permissions, credential owner, and expiry;
- decisions or actions it may take;
- applicable controls and required approvals;
- evaluation suites, current results, and monitored outcomes;
- cost budget and usage;
- incidents, known limitations, and change history;
- stop mechanism and retirement condition.

If nobody can list the agents that can modify customer, financial, code, or employee state, the company does not control them.

16.2 Divide responsibility without dividing accountability

The **business owner** defines the outcome, authority, and acceptable failure. **Agent Operations** provides registry, observability, evaluation infrastructure, deployment standards, and lifecycle controls. **Platform teams** operate shared models, tools, context services, and environments. **Security, risk, legal, and privacy** define specialist controls and challenge the deployment.

These roles can be held by a few people in a scale-up. What matters is that they are not collapsed into "the AI team." A technically reliable agent can still pursue the wrong business objective. A business owner cannot waive an enterprise access control by calling the experiment urgent.

NIST's Generative AI Profile organizes risk work across governance, mapping, measurement, and management throughout the lifecycle [14]. The practical lesson is to connect inventory, testing, monitoring, incident response, and ownership rather than treat them as launch paperwork.

16.3 Control the change surface

An agent can change even when its application code does not. The model provider updates a version. A context source changes. A tool adds a parameter. A policy is revised. Another agent begins consuming its output.

Version and test the combination that produces behavior. Material changes should run against relevant evaluations before release. Progressive rollout limits exposure. The registry should show which units and workflows depend on the changed component.

Multi-agent design increases this need. More agents may improve specialization, but every handoff creates another state boundary, failure mode, and cost. Use multiple agents because the task has a real separation of context or authority, not because a diagram looks sophisticated.

16.4 Run incidents as operating failures

Consider an **illustrative incident**. A sourcing agent reads a malicious instruction embedded in a supplier document. It calls a procurement tool using a broad shared token and changes payment details. A second agent treats the changed record as authoritative.

The root cause is not simply “prompt injection.” The organization exposed an unnecessary tool during document analysis, failed to bind the credential to a narrow resource and action, allowed one agent to write a state another trusted, and lacked an approval for a consequential change.

The response should revoke credentials, preserve traces, repair affected records, notify the right owners, and add the case to evaluation. The deeper correction narrows authority and separates untrusted reading from state-changing action.

Protocols such as MCP make tools easier to connect, which increases the importance of audience-bound credentials, consent, and resource-specific authorization [16]. Integration convenience is not a control model.

16.5 Retire what no longer earns its place

Agents should have sunset conditions. Usage alone is insufficient. An agent may be popular while generating review cost or relying on an outdated process. Compare outcome, total cost, risk, and maintenance with the current alternative. Remove credentials and integrations when the system is retired.

Agent Operations succeeds when the company can answer, quickly: Which agents can act? For whom? Using what? Under whose authority? With what evidence? And how do we stop them? At scale, those are operating questions, not technical inventory trivia.

17. The Manager After Coordination Gets Cheap

Management will not disappear because an agent can summarize status. Some management work should disappear.

A manager who collects updates, reformats them, and carries information between rooms is performing coordination that software can increasingly reduce. A manager who clarifies intent, designs decisions, resolves conflict, develops judgment, and repairs interfaces is doing work the organization needs more of as execution accelerates.

17.1 Five accountabilities

The manager in an adaptive company has five enduring accountabilities.

Set usable intent. Translate strategy into outcomes and constraints that a team can apply without asking for preference on every case.

Design decisions. Make ownership, evidence, authority, escalation, and recourse explicit. Delegation is incomplete until the receiver can make a valid choice the manager dislikes.

Maintain context. Ensure the team can reach current facts, policy, history, and customer reality. Do not become the private database that makes every decision possible.

Resolve exceptions and conflict. Handle the work that cannot be settled within existing interfaces. Then decide whether the exception should remain rare or trigger a redesign.

Develop judgment. Help people understand tradeoffs, not simply reproduce the manager's answer. Use agent traces and decision outcomes as material for coaching.

These accountabilities do not imply that every manager becomes an AI specialist. They imply that management shifts from moving information to improving the conditions under which decisions are made.

17.2 The Synthesizer

ATOM uses one specialized coordination role. A **Synthesizer** helps an outcome loop preserve coherence across intent, context, decisions, execution, and evidence. The role may be held by a product operations lead, chief of staff, program leader, or manager. It is not automatically a new job title.

The Synthesizer notices that the team is optimizing a proxy, that an exception lacks an owner, that two sources conflict, or that learning never enters the Playbook. The role convenes the right decision and improves the interface. It does not become the permanent human router for all work. If everything depends on the Synthesizer's memory, the role has recreated the debt it was meant to remove.

17.3 Building middle management

A company moving from forty to one hundred and fifty people needs managers because coordination, development, and local judgment become real work. Adding layers before defining their rights creates messengers. Avoiding layers while every conflict returns to the founder creates bottlenecks.

Start with recurring decisions and interfaces. Give a manager an outcome, visible economics, authority, context, and a route for exceptions. Decide which decisions belong to the business unit, shared function, and center. Then choose the span that allows the manager to perform the work, rather than importing a universal ratio.

Agents can reduce reporting and preparation load. They may allow a manager to support a wider span in stable work. In novel, physical, regulated, or relationship-heavy work, human attention may remain the constraint. The organization should change spans from observed work, not a forecast of automatic leverage.

17.4 Influence without hierarchy

The strategic COO increasingly works across peers who do not report to them. Authority comes from making the enterprise tradeoff legible.

Frame the contested decision, expose shared dependencies, show the evidence and uncertainty, state which right belongs where, and name the cost of delay. Ask a model to challenge the argument, surface stakeholder incentives, and generate plausible objections before the meeting. Use it to improve preparation, not simulate consensus.

Peer influence grows when the COO can separate enterprise constraints from personal preference. The goal is not to win every argument. It is to leave the company with a decision that others understand and can execute.

17.5 The founder's hardest delegation

The founder has given a business-unit leader a margin envelope. The leader chooses a price inside it. The founder believes another price would be better.

If the decision is sound, within authority, and supported by required evidence, taking it back teaches the organization that delegation ends at disagreement. The founder may challenge the reasoning, watch the outcome, and revise the envelope at the agreed review. They should not silently reclaim the case.

This is where personal standards can limit initiative. High standards are valuable; unstated standards enforced through intervention are not. The manager after cheap coordination is a steward of the decision system, including when the system produces an answer they would not have chosen.

18. Accountability Cannot Be Delegated

An agent sends the message. A manager approved the workflow. A vendor supplied the model. A platform team exposed the tool. A business unit defined the target.

When the outcome harms a customer, each party can accurately describe only part of the cause. The organization still owes an answer.

18.1 Separate five ideas

Execution is performing an action. **Authority** is legitimate permission to choose or act. **Responsibility** is an assigned duty within the system. **Accountability** is the obligation to explain, justify, and repair the outcome. **Liability** is the legal allocation of consequences.

An agent can execute and receive bounded authority. It cannot hold institutional accountability. A vendor contract may allocate liability without becoming the company's moral or managerial answer to an affected person.

18.2 Build the accountability chain

For each consequential agent-assisted outcome, the company should be able to trace:

1. the board or executive intent;
2. the business owner who chose the use and result;
3. the authority granted to the person, workflow, or agent;
4. the context, controls, and version in force;
5. the actual action and evidence;
6. the person or group affected;
7. the route to challenge, correction, and remedy.

The chain prevents a familiar form of evasion: the model made the decision. Models produce outputs inside systems that people select, connect, authorize, and operate.

For uses covered by law, specific duties may attach to providers and deployers. The EU AI Act, for example, requires deployers of high-risk systems to assign human oversight to people with the necessary competence, authority, and support, alongside other technical and organizational measures [18]. The exact legal duty depends on role and use. The operating principle is broader: oversight without authority is ceremonial.

18.3 Recourse is part of the design

An affected person needs more than disclosure that AI was involved. They need a way to contest the relevant result, reach someone able to change it, and receive repair when warranted.

The route should match the consequence. A poor recommendation may need feedback. A denied service, employment action, financial decision, or binding contract may require explanation, preserved evidence, independent review, and timely correction. The person handling the appeal should not merely ask the same system again.

Recourse also improves the operating model. Challenges reveal missing context, harmful proxies, weak policy, and cases the evaluation suite did not contain. The organization should connect them to learning without using affected people as unpaid testers.

18.4 Illustrative failure review

A hiring-screening agent ranks a candidate below the interview threshold. A recruiter follows the recommendation. Months later, an audit finds that the agent inferred seniority from job-title patterns that disadvantaged candidates from smaller companies.

The failure review should not end with the recruiter or model. The business owner chose automated ranking. The data owner approved the inputs. The agent owner defined the evaluation. The control owner approved the authority boundary. Executive intent prioritized processing speed without an adequate fairness and recourse measure.

The company pauses the action, identifies affected decisions, offers review, repairs the evaluation set, changes the allowed features, and reconsiders whether ranking should carry that authority. Accountability moves upward to the people able to change the system and outward to those who experienced the result.

18.5 Capability does not dissolve obligation

Future agents may operate for longer periods, coordinate with other agents, and manage larger budgets. If capability rises, companies may move human attention from individual actions to authority design, monitoring, incident response, and periodic review.

One condition must remain true: a named human institution can explain why the system was permitted to act and can intervene on behalf of those affected. If nobody can do so because autonomy is distributed across vendors, agents, and policies, the organization has not transcended management. It has abandoned accountability.

Companies may delegate action to software. They cannot delegate the duty to justify, repair, and answer for the consequences.

Conclusion: Build One Honest Loop

The adaptive company does not begin with an announcement.

It begins when an operating leader chooses one consequential decision that repeatedly returns, waits, or fails. The company makes the intended outcome explicit. It assembles authorized context. It states who may decide and within what boundary. It assigns work to people, rules, workflows, and agents. It requires evidence of the path and the result. It gives learning to owners who can change the right part of the system.

Then it watches what happens.

The first loop will reveal uncomfortable facts. Delegation may be nominal. A trusted data source may be stale. A shared function may be funded to enforce policy but not to provide usable service. An agent may remove work from one team and create review elsewhere. A founder's intervention may remain the fastest short-term answer and the largest long-term constraint.

That evidence is the work.

Do not begin with an enterprise autonomy target. Do not count generated artifacts as value. Do not build a central AI organization that owns everybody else's outcomes. Do not use observability to rank employees or governance to disguise fear of delegation.

Build one loop whose result matters. Bound the consequence. Keep the traces. Let a real owner make a real decision. When the result is poor, repair the context, right, interface, control, or assumption that produced it. When it is good, prove that normal operations can sustain it before expanding.

ATOM is a model, not the destination. Its value lies in the questions it forces the company to answer:

- What outcome are we trying to create?
- What context is authorized and current?
- Who or what may commit us?
- What can act, and what must stop?
- What evidence will prove the result?
- Who may change the system?
- Who remains accountable?

Answer those questions for one recurring decision and the company has begun to change its operating system.

Build one honest loop. Learn from it. Then decide what deserves to repeat.

Appendices

Field Kit

The following compact templates are designed for use with an approved AI assistant and the company's existing tools. Adapt the depth to the consequence of the work.

1. ATOM loop canvas

Layer	Complete this statement
Intent	We seek ___ by ___ while protecting ___
Context	The required sources are ; <i>owners/freshness are</i>
Decision	___ may decide ___ within ; <i>escalate when</i>
Execution	People do ; <i>rules do</i> ; agents do ___
Evidence	We retain ___ and measure the outcome through ___
Learning	___ reviews ___ and may change ___
Control plane	Identity, permissions, approval, stop, rollback, and recourse are ___

2. Operational-debt analysis prompt

Analyze the attached anonymized workflow records. Reconstruct trigger, path, waiting, handoffs, missing context, escalations, reversals, exceptions, controls, and outcome. Separate observed evidence from inference. Link every pattern to source cases. Propose a metric, owner, repayment experiment, and alert for each material liability. Do not rank employees or invent unavailable data.

3. Decision specification

Field	Definition
Decision	The commitment being made, not the meeting
Outcome protected	Customer, economic, quality, risk, or strategic result
Accountable owner	Named human role
Inputs	Required facts and authorized sources
Criteria	Tradeoffs, thresholds, and prohibited conditions
Consequence	Severity, scale, and affected parties
Reversibility	How and at what cost the action can be undone
Authority	Human, rule, workflow, and agent rights
Evidence	Rationale, sources, action, approval, and outcome
Escalation/recourse	Trigger, destination, response expectation
Review	When the decision design expires or changes

4. Agent autonomy matrix

Level	Grant	Required evidence
Assist	Retrieve, summarize, draft	Sources and output
Propose	Recommend an action	Options, rationale, uncertainty
Prepare	Create the exact action for approval	Full parameters and checks
Act and report	Execute reversible action inside policy	Trace, result, notification
Act within budget	Manage bounded multi-step work	Continuous controls, outcome, exceptions

Increase authority by action and condition, never by giving the agent a general promotion.

5. Context envelope

Record task state, enterprise facts, policy, relevant history, provenance, and access. For each source add owner, scope, freshness, precedence, sensitive fields, and behavior when missing or conflicting.

6. Pilot selection card

Compare value, frequency, latency, evaluability, context readiness, reversibility, consequence, integration load, ownership, and transfer to the run. Reject a candidate that cannot produce observable learning even if the demonstration is impressive.

7. First 30 days

- Days 1-5: charter, historical trace, baseline, outcome, kill condition.
- Days 6-10: decision design, context envelope, handoff contracts.
- Days 11-15: test environment, permissions, logs, evaluations, failure rehearsal.
- Days 16-20: shadow operation and calibrated review.
- Days 21-25: limited release with pause and escalation.
- Days 26-30: expand, revise, contain, or stop.

8. Evidence scorecard

Review outcome, path time, handoffs, human intervention, reversals, failure severity, recovery, model/tool cost, maintenance, affected-party feedback, and evidence completeness. Segment results and inspect tail cases.

9. Fractal Unit contract

State purpose, economics, inputs, outputs, decisions, dependencies, context, controls, evidence, learning rights, and lifecycle. Mark each element as local choice, shared interface, or enterprise invariant.

10. Agent registry

Record owner, purpose, version, environment, models, tools, sources, dependencies, identity, permissions, actions, controls, evaluations, outcomes, cost, incidents, stop mechanism, and retirement condition.

11. Ninety-day gate

Choose one: expand, revise, contain, or stop. Record the outcome evidence, complete cost, risk, distribution of effects, operating owner, unresolved assumptions, and exact authority or scope change.

12. Quality gate

Before expanding a loop, score originality of the design, clarity, evidence, practical usefulness, specificity, flow, internal consistency, and absence of inflated AI language. Any weak dimension requires a named revision, not a lower standard.

Bibliography

- [1] Stanford Institute for Human-Centered Artificial Intelligence, “Artificial Intelligence Index Report 2025,” 2025. Accessed: Sep. 11, 2026. [Online]. Available: https://hai.stanford.edu/assets/files/hai_ai-index-report-2025_chapter1_final.pdf
- [2] B. Cottier, B. Snodin, D. Owen, and T. Adamczewski, “LLM Inference Price Trends.” Accessed: Sep. 11, 2026. [Online]. Available: <https://epoch.ai/data-insights/llm-inference-price-trends>
- [3] J. Gans, *The Microeconomics of Artificial Intelligence*. The MIT Press, 2025. doi: [10.7551/mitpress/15248.001.0001](https://doi.org/10.7551/mitpress/15248.001.0001).
- [4] Model Evaluation & Threat Research, “Task-Completion Time Horizons of Frontier AI Models,” 2026. Accessed: Sep. 11, 2026. [Online]. Available: <https://metr.org/time-horizons/>
- [5] Stanford Institute for Human-Centered Artificial Intelligence, “The 2026 AI Index Report,” Apr. 2026. Accessed: Sep. 11, 2026. [Online]. Available: <https://hai.stanford.edu/ai-index/2026-ai-index-report>
- [6] E. Brynjolfsson, D. Li, and L. R. Raymond, “Generative AI at Work,” *The Quarterly Journal of Economics*, vol. 140, no. 2, 2025, doi: [10.1093/qje/qjae044](https://doi.org/10.1093/qje/qjae044).
- [7] J. Becker, N. Rush, B. Barnes, and D. Rein, “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity,” Jul. 2025. Accessed: Sep. 11, 2026. [Online]. Available: https://metr.org/Early_2025_AI_Experienced_OS_Devs_Study-paper.pdf
- [8] Model Evaluation & Threat Research, “Update: How Much Does AI Speed Up Experienced Developers?” Accessed: Sep. 11, 2026. [Online]. Available: <https://metr.org/blog/2026-02-24-uplift-update/>
- [9] DORA, “State of AI-Assisted Software Development 2025,” 2025. Accessed: Sep. 11, 2026. [Online]. Available: <https://dora.dev/research/2025/dora-report/>
- [10] Anthropic, “Demystifying Evals for AI Agents.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- [11] Intercom, “Fin AI Agent Reporting.” Accessed: Sep. 12, 2026. [Online]. Available: <https://www.intercom.com/help/en/articles/7837533-fin-ai-agent-reporting>
- [12] LangChain, “Discover Errors and Usage Patterns with Insights.” Accessed: Sep. 12, 2026. [Online]. Available: <https://docs.langchain.com/langsmith/insights>
- [13] E. Dillon, S. Jaffe, N. Immorlica, and C. Stanton, “Shifting Work Patterns with Generative AI,” Apr. 2025, Accessed: Sep. 12, 2026. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/shifting-work-patterns-with-generative-ai/>
- [14] C. Autio *et al.*, “Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile,” NIST AI 600–1, Jul. 2024. doi: [10.6028/NIST.AI.600-1](https://doi.org/10.6028/NIST.AI.600-1).
- [15] OpenAI, “A Practical Guide to Building Agents.” Accessed: Sep. 12, 2026. [Online]. Available: <https://openai.com/business/guides-and-resources/a-practical-guide-to-building-ai-agents/>
- [16] Model Context Protocol, “Authorization.” Accessed: Sep. 12, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-06-18/basic/authorization>
- [17] European Commission, “AI Act.” Accessed: Sep. 12, 2026. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai>
- [18] European Parliament and Council of the European Union, “Regulation (EU) 2024/1689 Laying Down Harmonised Rules on Artificial Intelligence, Consolidated Version.” Accessed: Sep. 12, 2026. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1689/2026-07-27/eng>